

Laboratorio de Simulación de Phishing en Entorno Controlado



Laboratorio de Simulación de Phishing en Entorno Controlado	1
1. Introducción	3
1.2 Qué es el phishing	3
1.3 Impacto del phishing en las organizaciones	4
1.4 Justificación del proyecto	4
1.5 Objetivos	5
1.5.1 Objetivo general	5
1.5.2 Objetivos específicos	5
2. Marco Teórico	5
2.1 Ingeniería social	5
2.2 Tipologías de phishing	5
2.2.1 Phishing tradicional	6
2.2.2 Spear phishing	6
2.2.3 Whaling	6
2.2.4 Smishing	7
2.2.5 Vishing	7
2.2.6 Spoofing	7
2.2.7 Evil Twin	7
2.2.8 Quishing	7
2.3 Ciclo de una campaña de phishing	8
2.4 Indicadores de detección	8
2.4.1 Señales en el mensaje	8
2.4.2 Señales técnicas	10
2.5 Medidas defensivas	11
2.5.1 Medidas técnicas	11
3.5.2 Medidas organizativas	12
3.5.3 Medidas personales (usuario final)	12
3. Diseño del Laboratorio	12
3.1 Requisitos, asignación de recursos y función de cada máquina virtual	13
3.2 Arquitectura del entorno	14
3.3 Segmentación de red	14
3.4 Metodología de trabajo	15
3.5 Direccionamiento IP	15
4. Implementación del Entorno Virtual	15
4.1 Creación de la red del laboratorio	16
4.2 Instalación del servidor GoPhish	16
4.3 Instalación de Ubuntu Server (Servidor de correo)	19
4.3.1 Instalación de Postfix	19
4.3.2 Instalación y configuración de Dovecot	22
4.4 Configuración de la red	24
4.4.1 Configuración de las interfaces de red.	24
4.4.2 Configuración del acceso remoto mediante SSH	25
4.4.3 Configuración de direcciones IP estáticas	26
4.4.4 Verificación de la conectividad	26

4.5 Instalación de Windows 10 (Equipo Cliente)	27
4.6 Configuración del cliente de correo	30
4.6.1 Creación del usuario de correo en el servidor	30
4.6.2 Instalación de Thunderbird	31
4.6.3 Configuración de la cuenta	32
4.6.4 Prueba de funcionamiento	33
4.7 Instalación y configuración de GoPhish	33
4.7.1 Instalación de GoPhish	34
4.7.2 Configuración del perfil de envío (SMTP)	36
4.7.3 Creación de la plantilla de correo	36
4.7.6 Creación de la campaña y prueba de funcionamiento	41
5. Conclusiones	43
5.1 Objetivos alcanzados	43
5.2 Conocimientos adquiridos	44
5.3 Trabajo futuro	44

1. Introducción

1.2 Qué es el phishing

El phishing es una técnica de ingeniería social utilizada en ciberseguridad ofensiva que consiste en la suplantación de identidad con el objetivo de engañar a un usuario para que revele información sensible, como credenciales, datos personales o acceso a sistemas.

Este tipo de ataque se basa principalmente en la manipulación psicológica del usuario, más que en vulnerabilidades técnicas del sistema. Los atacantes suelen aprovechar factores como la urgencia, la autoridad o el miedo para inducir a la víctima a tomar decisiones precipitadas sin verificar la legitimidad del mensaje recibido.



1.3 Impacto del phishing en las organizaciones

El phishing es una de las amenazas más frecuentes y dañinas en entornos corporativos, ya que actúa como puerta de entrada a incidentes de seguridad más graves, como el robo de información, el acceso no autorizado a sistemas o la propagación de malware.

Entre sus principales impactos se encuentran:

- Pérdida de información confidencial
- Compromiso de credenciales corporativas
- Interrupción de servicios
- Daño reputacional
- Pérdidas económicas directas e indirectas

Además, el phishing suele ser el primer paso en ataques más complejos como ransomware o intrusiones persistentes en redes corporativas.



1.4 Justificación del proyecto

Este proyecto nace con la idea de entender de forma práctica cómo funcionan las campañas de phishing, pero en un entorno seguro y controlado. La intención es poder ver no solo la parte técnica del ataque, sino también cómo pueden reaccionar las personas ante este tipo de correos.

Para ello, se ha creado un laboratorio virtual que permite simular situaciones reales sin ningún riesgo, con el objetivo de observar cómo se comportan tanto los sistemas como los usuarios cuando reciben mensajes de phishing simulados.

Lo interesante de este trabajo es que el phishing no se puede comprender del todo solo desde la teoría. Hace falta verlo en acción para poder reconocer sus patrones, entender el impacto que puede tener y pensar mejor en cómo defenderse de este tipo de ataques.

1.5 Objetivos

1.5.1 Objetivo general

La idea de este proyecto es diseñar y montar un laboratorio virtual que permita simular campañas de phishing de forma controlada. Con esto, se busca poder entender mejor cómo funcionan este tipo de ataques en la práctica y analizar qué técnicas pueden ayudar a detectarlos y reducir su impacto.

1.5.2 Objetivos específicos

Diseñar una arquitectura de red aislada basada en máquinas virtuales, que permite trabajar en un entorno seguro para las pruebas. Dentro de este laboratorio se ha implementado un

servidor de correo interno y una plataforma de simulación de phishing, con el objetivo de recrear situaciones lo más realistas posible.

2. Marco Teórico

2.1 Ingeniería social

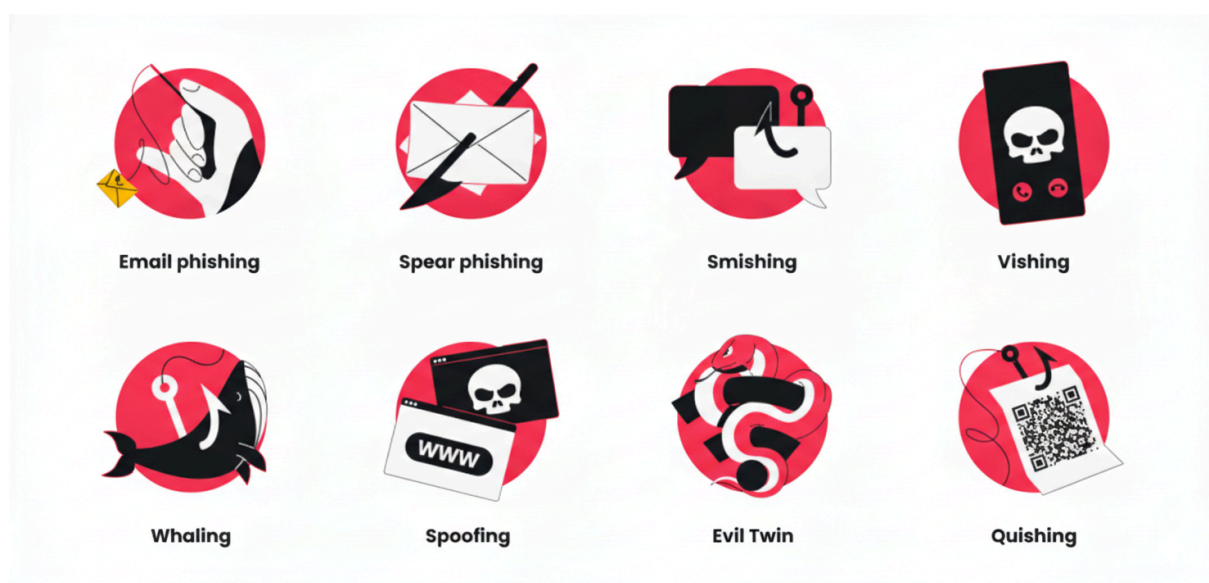
La ingeniería social es un conjunto de técnicas que buscan influir o engañar a las personas para conseguir información confidencial o que realicen alguna acción concreta. A diferencia de los ataques puramente técnicos, aquí el foco no está en los sistemas, sino en las personas, que suelen ser el punto más vulnerable de la seguridad.

En ciberseguridad, este tipo de ataques se consideran especialmente peligrosos porque no dependen de fallos en el software, sino de factores humanos como la confianza, la distracción o la falta de concienciación.

Dentro de este contexto, el phishing es una de las formas más habituales de ingeniería social. Se basa en el uso de medios digitales, sobre todo el correo electrónico, para engañar a las víctimas y conseguir que revelen información o realicen acciones sin darse cuenta del riesgo.

2.2 Tipologías de phishing

El phishing puede presentarse de distintas maneras, dependiendo del objetivo del atacante, el canal que se utilice y el nivel de personalización del engaño.



2.2.1 Phishing tradicional

Es la forma más habitual. Se basa en el envío masivo de correos electrónicos fraudulentos que imitan a organizaciones legítimas, con la intención de engañar al mayor número posible de usuarios.

Este tipo de ataques suele caracterizarse por un gran volumen de envíos, poca o nula personalización y el uso de mensajes bastante genéricos. A menudo incluyen enlaces o archivos maliciosos diseñados para comprometer al usuario.

2.2.2 Spear phishing

Este tipo de ataque se dirige a una persona concreta o a un grupo muy reducido. Antes de lanzar el engaño, el atacante suele investigar a la víctima para poder personalizar el mensaje y hacer que resulte más creíble.

En este caso, se trata de ataques mucho más elaborados, donde se utiliza información real de la persona para aumentar las posibilidades de éxito.

Se caracteriza por un alto nivel de personalización, el uso de datos reales de la víctima y una mayor credibilidad en el contenido del mensaje.

2.2.3 Whaling

Es una variante del *spear phishing* que se dirige a personas con altos cargos dentro de una organización, como directivos, responsables financieros o administradores de sistemas.

El objetivo suele ser acceder a información crítica, cometer fraudes económicos o conseguir una escalada de privilegios dentro del sistema.

2.2.4 Smishing

En este caso, los ataques de phishing se realizan a través de SMS o aplicaciones de mensajería instantánea.

Suelen apoyarse en enlaces acortados, mensajes urgentes y la suplantación de servicios conocidos para intentar que la víctima actúe sin pensar demasiado.

2.2.5 Vishing

El *vishing* se lleva a cabo mediante llamadas telefónicas. El atacante se hace pasar por una entidad de confianza, como soporte técnico, un banco o un servicio corporativo.

Su objetivo es obtener información sensible directamente del usuario durante la conversación.

2.2.6 Spoofing

El *spoofing* es una técnica de suplantación en la que el atacante falsifica la identidad de un sistema, dispositivo o servicio para hacerse pasar por una fuente legítima.

Puede aparecer en distintos niveles, como el correo electrónico, direcciones IP o DNS. En el contexto del phishing, se utiliza sobre todo para que el mensaje parezca provenir de un remitente confiable.

2.2.7 Evil Twin

Este ataque consiste en crear un punto de acceso WiFi falso que imita a uno legítimo.

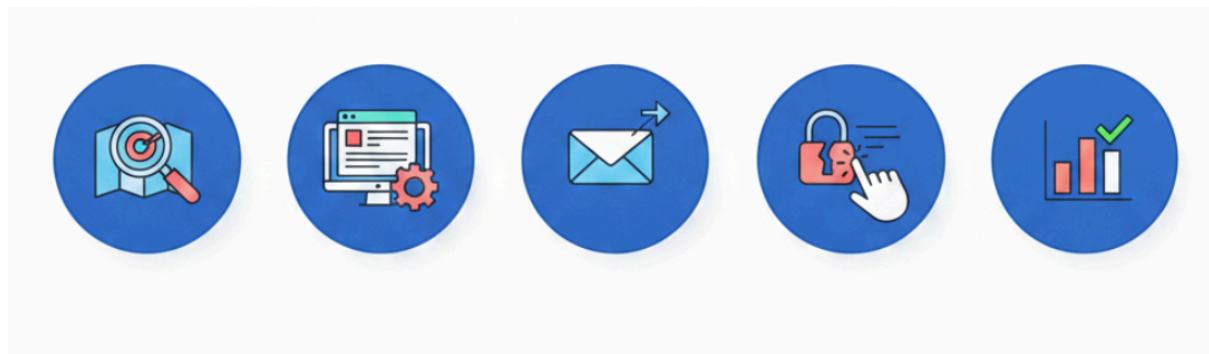
El objetivo es engañar a los usuarios para que se conecten a esa red y así poder interceptar su tráfico, capturar credenciales o monitorizar su actividad. Es especialmente peligroso en redes públicas como cafeterías, aeropuertos o bibliotecas.

2.2.8 Quishing

El *quishing* es una variante del phishing que utiliza códigos QR maliciosos.

Al escanearlos, la víctima puede ser redirigida a páginas falsas, formularios diseñados para robar credenciales o incluso descargas de malware. Es especialmente complicado de detectar, ya que el usuario no ve directamente la URL a la que está accediendo.

2.3 Ciclo de una campaña de phishing



Una campaña de phishing suele seguir un ciclo estructurado que permite al atacante maximizar su efectividad:

1. Planificación

Se define el objetivo del ataque y se elige el tipo de estrategia que se va a utilizar.

2. Preparación

Se crea toda la infraestructura necesaria: correos falsos, páginas web fraudulentas y otros recursos.

3. Ejecución

Se envían los mensajes a las posibles víctimas seleccionadas.

4. Interacción

La víctima abre el correo y realiza alguna acción, como hacer clic en un enlace, introducir datos o descargar un archivo.

5. Explotación

El atacante obtiene la información sensible o las credenciales de la víctima.

6. Análisis

Se revisan los resultados de la campaña para ver qué ha funcionado y ajustar futuras acciones.

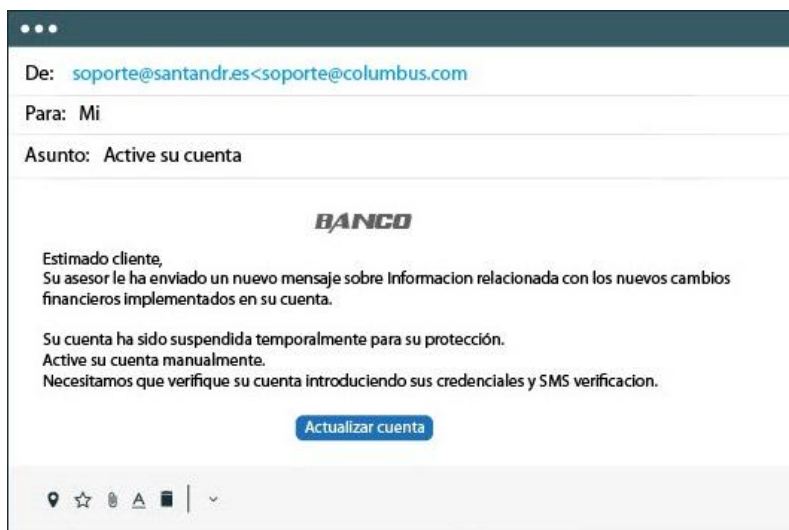
2.4 Indicadores de detección

Detectar un ataque de phishing consiste en fijarse en señales que pueden parecer pequeñas, pero que en conjunto delatan que algo no es normal. Estas señales pueden aparecer tanto en el contenido del mensaje como en su comportamiento técnico.

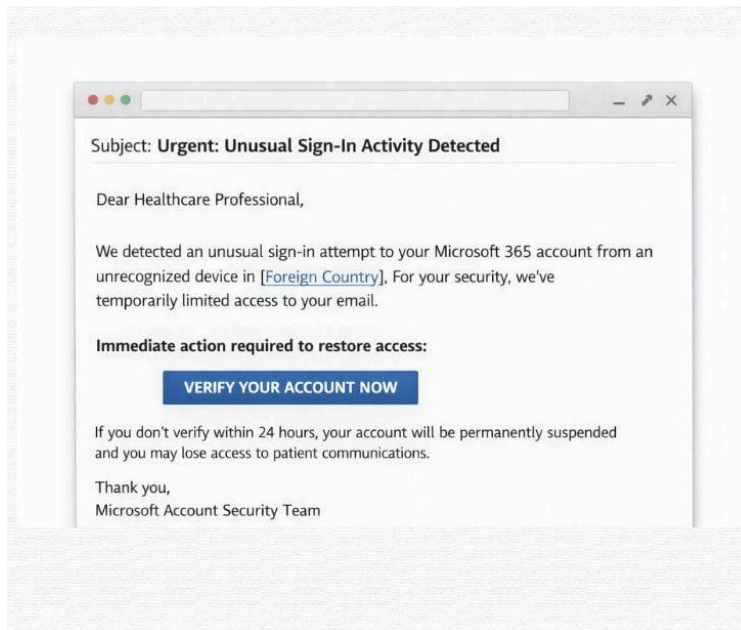
2.4.1 Señales en el mensaje

A nivel de usuario, hay varios indicios bastante habituales que pueden levantar sospechas:

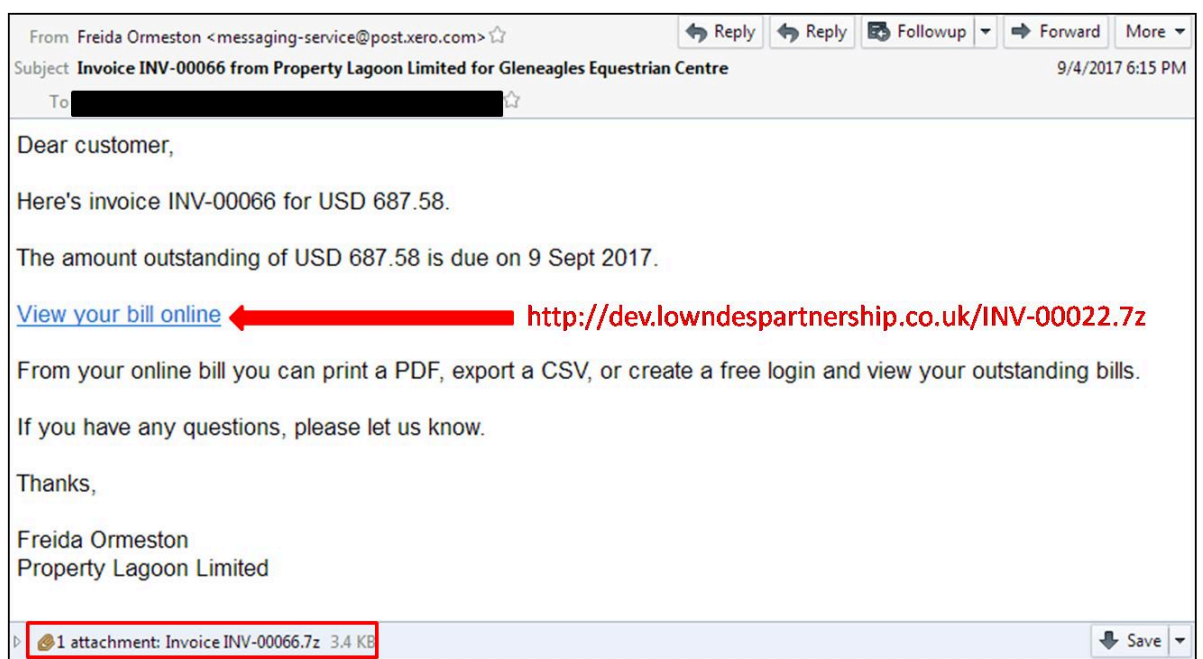
- Correos que no provienen de direcciones oficiales o que están ligeramente modificadas



- Errores de escritura o frases mal construidas
- Mensajes que intentan generar urgencia o presión para actuar rápido



- Enlaces que no coinciden con la página real a la que dicen llevar
- Archivos adjuntos que no se esperan



- Peticiones de contraseñas o datos personales, algo que normalmente no se solicita por correo

2.4.2 Señales técnicas

También existen pistas más técnicas que ayudan a detectar este tipo de ataques:

- Cabeceras de correo con información sospechosa o incoherente

```
ARC-Authentication-Results: i=1; mx.google.com
  dkim=pass header.i=<legitimate university email account> header.s=default
  header.b=pUMk3rBl;
  spf=pass (google.com: domain of <legitimate university email host domain>
  designates <IP address associated with university email host domain> as
  permitted sender) smtp.mailfrom=<legitimate university email host domain>;
  dmarc=fail (p=none sp=none dis=none ) header.from=<spoofed domain of think tank>
...
Received: from evanger8 by box2239.bluehost.com with local (Exim 4.96.2)
  (envelope-from <evanger8@box2239.bluehost.com>) id IrkTlk-OOOEeX-36 for <expert's
  receiving email>; Mon, 01 Jan 2024 18:40:37 -0700
To: <expert's receiving email>
Subject: [Invitation]
X-PHP-Script: <legitimate university email account>/move/send/fooe.php for 59.6.130.254,
59.6.130.254 X-PHP-Originating-Script: 1171:mail.php
MIME-Version: 1.0
Content-Type: text/html; charset=utf-8 Content-Transfer-Encoding: quoted-printable
From: <email address of legitimate think tank>
Cc:
Reply-To: <name of legitimate think tank staff> <spoofed account of legitimate think tank staff>
```

- Servidores de origen que no encajan con el remitente real
- Tráfico de red que no sigue patrones habituales

2.5 Medidas defensivas



La defensa frente al phishing no depende de una única solución, sino de combinar tecnología, buenas prácticas y concienciación de los usuarios. Como este tipo de ataques se basa sobre todo en la manipulación de las personas, es clave trabajar tanto la prevención como la detección y la respuesta ante incidentes.

2.5.1 Medidas técnicas

Estas medidas buscan reducir la probabilidad de que el ataque llegue al usuario o tenga éxito:

- **Filtros antispam y antiphishing:** sistemas que analizan los correos entrantes para detectar patrones sospechosos, dominios maliciosos o contenido fraudulento.
- **Autenticación multifactor (MFA):** añade una capa extra de seguridad, de modo que aunque una contraseña sea robada, no sea suficiente para acceder.
- **Protocolos de autenticación de correo (SPF, DKIM y DMARC):** ayudan a verificar si un correo realmente proviene del dominio que dice ser.
- **Sistemas IDS/IPS:** monitorizan el tráfico de red para detectar comportamientos anómalos o potencialmente maliciosos.
- **Análisis de enlaces y adjuntos:** inspección automática de URLs y archivos antes de que el usuario los abra.

3.5.2 Medidas organizativas

Estas dependen más de las políticas y procedimientos dentro de una organización:

- **Formación continua en ciberseguridad:** ayudar a los usuarios a reconocer intentos de phishing y mejorar su criterio.
- **Simulaciones de phishing controladas:** campañas internas que permiten evaluar cómo reaccionan los usuarios ante correos simulados.
- **Políticas de acceso:** aplicar el principio de mínimo privilegio para reducir el impacto si una cuenta se ve comprometida.
- **Protocolos de verificación:** establecer pasos claros para confirmar acciones sensibles, como transferencias o cambios de credenciales.
- **Canales de reporte:** facilitar mecanismos rápidos para informar correos o incidentes sospechosos.

3.5.3 Medidas personales (usuario final)

A nivel individual, también hay hábitos sencillos que marcan mucha diferencia:

- Revisar siempre el remitente antes de interactuar con un correo.
- No abrir enlaces o archivos de mensajes inesperados.
- Desconfiar de mensajes que generen urgencia o presión.
- Comprobar manualmente las URLs antes de introducir datos sensibles.
- Reportar cualquier correo sospechoso al equipo de IT o seguridad.

3. Diseño del Laboratorio

3.1 Requisitos, asignación de recursos y función de cada máquina virtual

Para la implementación del laboratorio de simulación de phishing se ha utilizado un entorno de virtualización, lo que permite ejecutar todos los componentes en un único equipo físico de forma aislada y controlada.

Se tiene como objetivo reproducir un entorno realista optimizando los recursos disponibles en el sistema anfitrión.

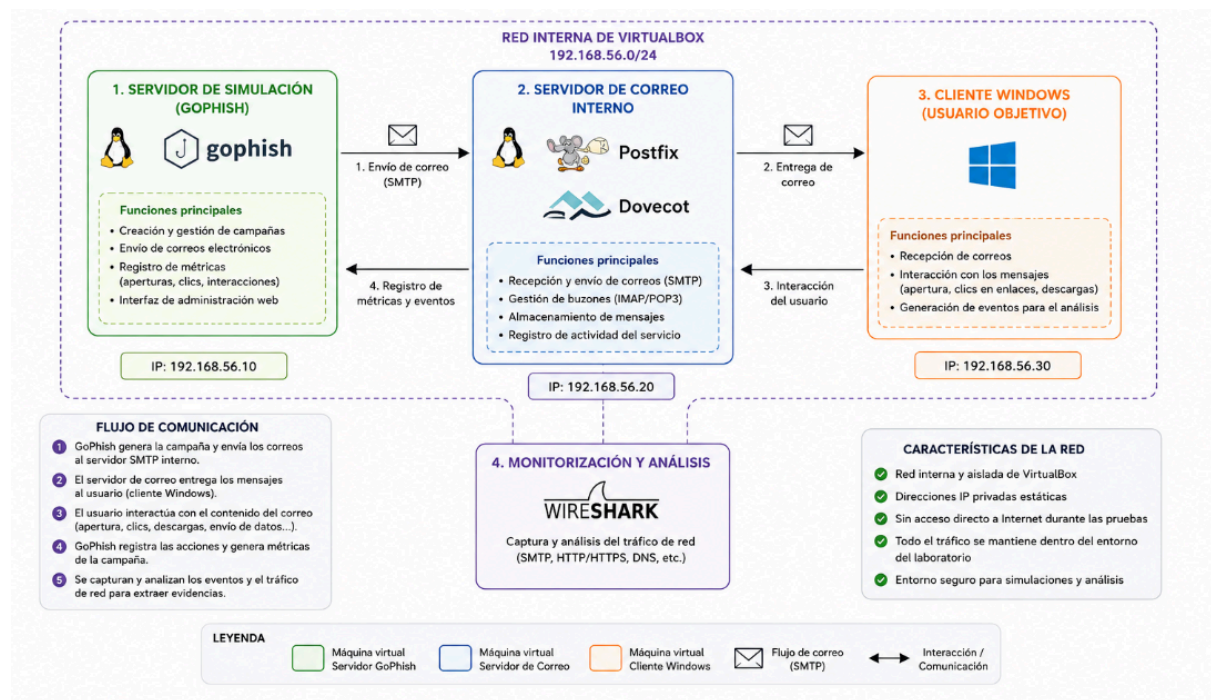
Máquina virtual	Sistema operativo	RAM	CPU	Almacenamiento	Función
Servidor GoPhish	Ubuntu Server	2 GB	2	20 GB	Gestión de campañas de phishing simuladas, envío de correos y registro de métricas mediante interfaz web.
Servidor de correo	Ubuntu Server	2 GB	2	20 GB	Gestión del servicio SMTP/IMAP (Postfix y Dovecot), recepción y entrega de correos dentro del entorno aislado.
Cliente Windows 10	Windows 10	3 GB	2	40 GB	Máquina víctima del laboratorio utilizada para recibir correos, interactuar con los escenarios y generar evidencias.

Las aplicaciones utilizadas serán:

- **VirtualBox** como plataforma de virtualización.
- **Ubuntu Server** como sistema operativo para los servidores del laboratorio.
- **Windows 10** como sistema cliente.
- **GoPhish** como herramienta de simulación de campañas de phishing.
- **Postfix** como servidor de correo saliente (SMTP).
- **Dovecot** como servidor de recepción de correo.
- **Wireshark** para el análisis del tráfico de red.

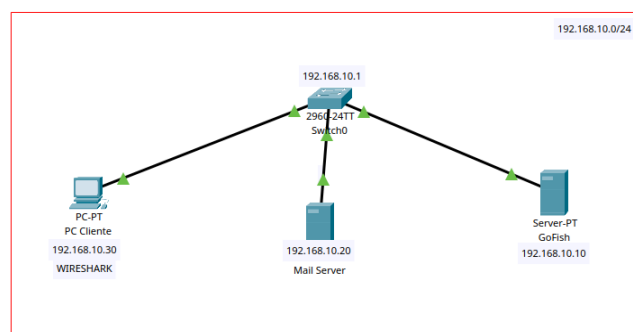
3.2 Arquitectura del entorno

La arquitectura se basa en un entorno virtualizado en VirtualBox y aislado en una red interna, se compone por tres máquinas virtuales, un servidor de simulación de campañas (Gophish), que crea y envía los correos y recoge los resultados, un servidor de correo que será el intermediario a la hora de entregar los emails y un cliente windows que será el usuario final.



3.3 Segmentación de red

Al tratarse de un laboratorio de estudio, para garantizar un entorno controlado, se ha optado por una red aislada y segura, evitando cualquier comunicación externa.



3.4 Metodología de trabajo

El laboratorio se montará paso a paso, comprobando que cada parte funciona correctamente antes de pasar a la siguiente.

Primero se crearán las tres máquinas virtuales y se configurará la red interna de VirtualBox. Después se instalarán los dos servidores Ubuntu y el equipo con Windows 10.

A continuación, se configurará el servidor de correo con Postfix y Dovecot. Después se instalará GoPhish y se conectará al servidor de correo para poder enviar las campañas de phishing simuladas.

Por último, se configurará el cliente de correo en Windows 10 y se realizarán varias pruebas para comprobar que todo funciona correctamente.

Durante todo el proceso se irán realizando capturas de pantalla para documentar la instalación y las pruebas realizadas.

3.5 Direccionamiento IP

Para que las máquinas virtuales puedan comunicarse entre sí, se utilizarán direcciones IP estáticas dentro de una red privada creada en VirtualBox. De esta forma, no será necesario un servidor DHCP y el laboratorio permanecerá aislado de la red externa.

Cada máquina virtual tendrá una dirección IP fija según la función que desempeñe. Una vez terminado el laboratorio, se documentarán las direcciones IP asignadas a cada equipo y la máscara de red utilizada.

4. Implementación del Entorno Virtual

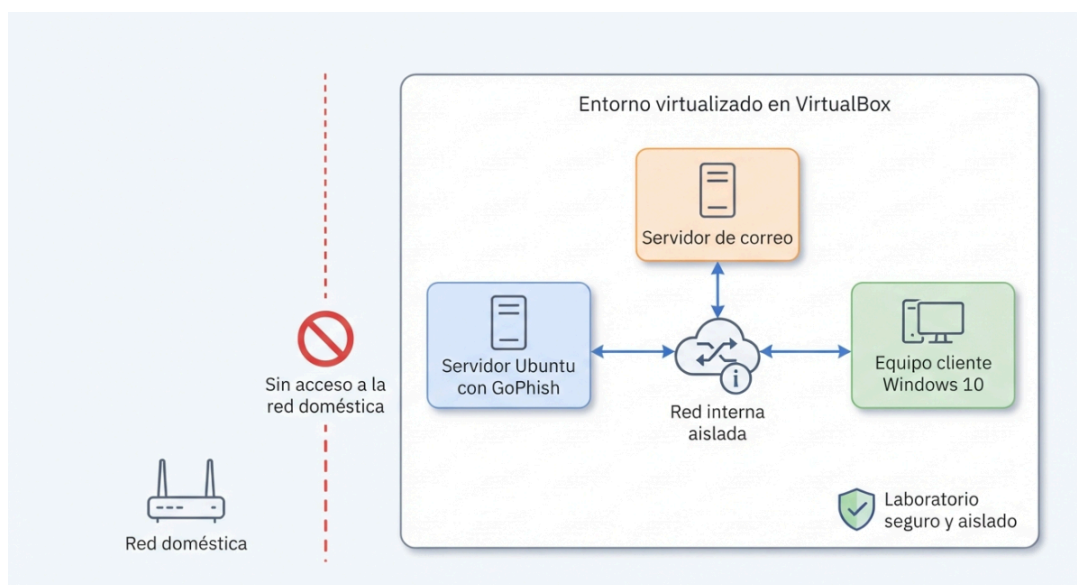
En este capítulo se explica cómo se ha montado el laboratorio para simular campañas de phishing en un entorno controlado. Se describen la instalación y la configuración de cada uno de los componentes, desde VirtualBox hasta las máquinas virtuales y los servicios necesarios.

El objetivo es disponer de un entorno seguro y completamente aislado donde poder realizar las pruebas de phishing sin afectar a equipos o redes reales. Para ello, cada componente se instalará y comprobará antes de continuar con el siguiente.

4.1 Creación de la red del laboratorio

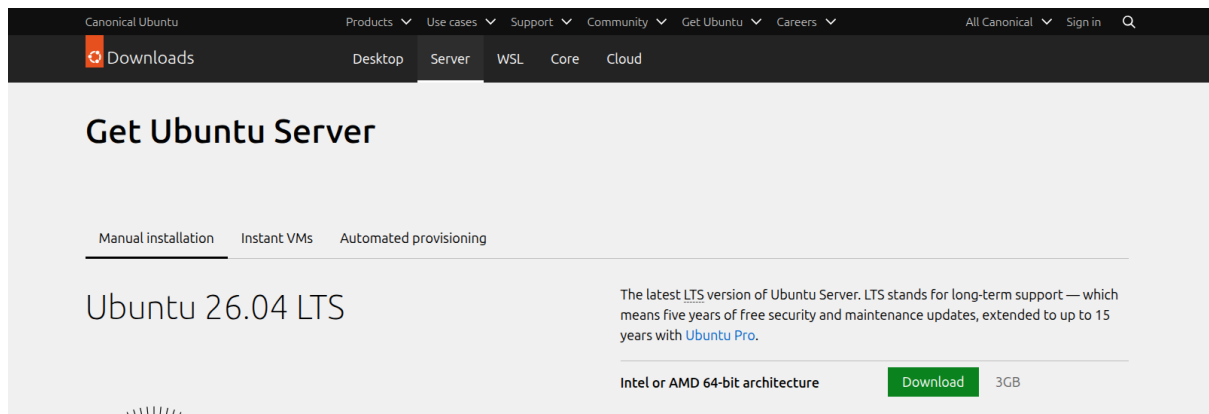
Para que las máquinas virtuales pudieran comunicarse entre sí sin acceder a la red doméstica, se creó una red interna en VirtualBox. De esta forma, todo el tráfico queda aislado y las pruebas de phishing se realizan en un entorno seguro, sin afectar a otros equipos.

Esta red será utilizada por las tres máquinas virtuales del laboratorio: el servidor con GoPhish, el servidor de correo y el equipo cliente con Windows 10. Más adelante, a cada una de ellas se le asignará una dirección IP fija para facilitar la comunicación entre los distintos servicios.

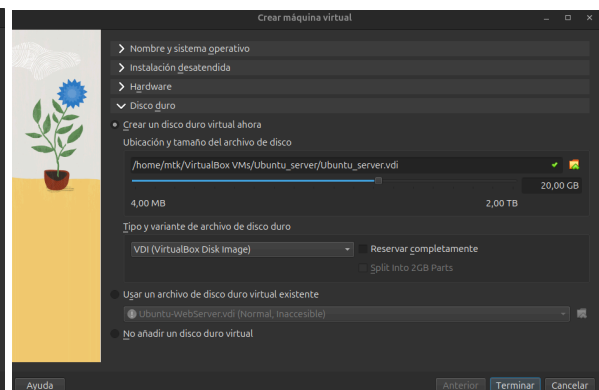
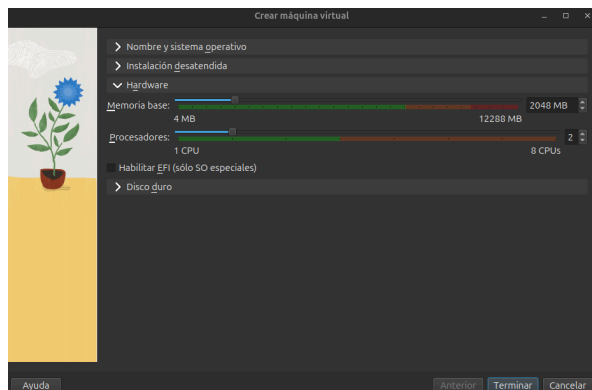
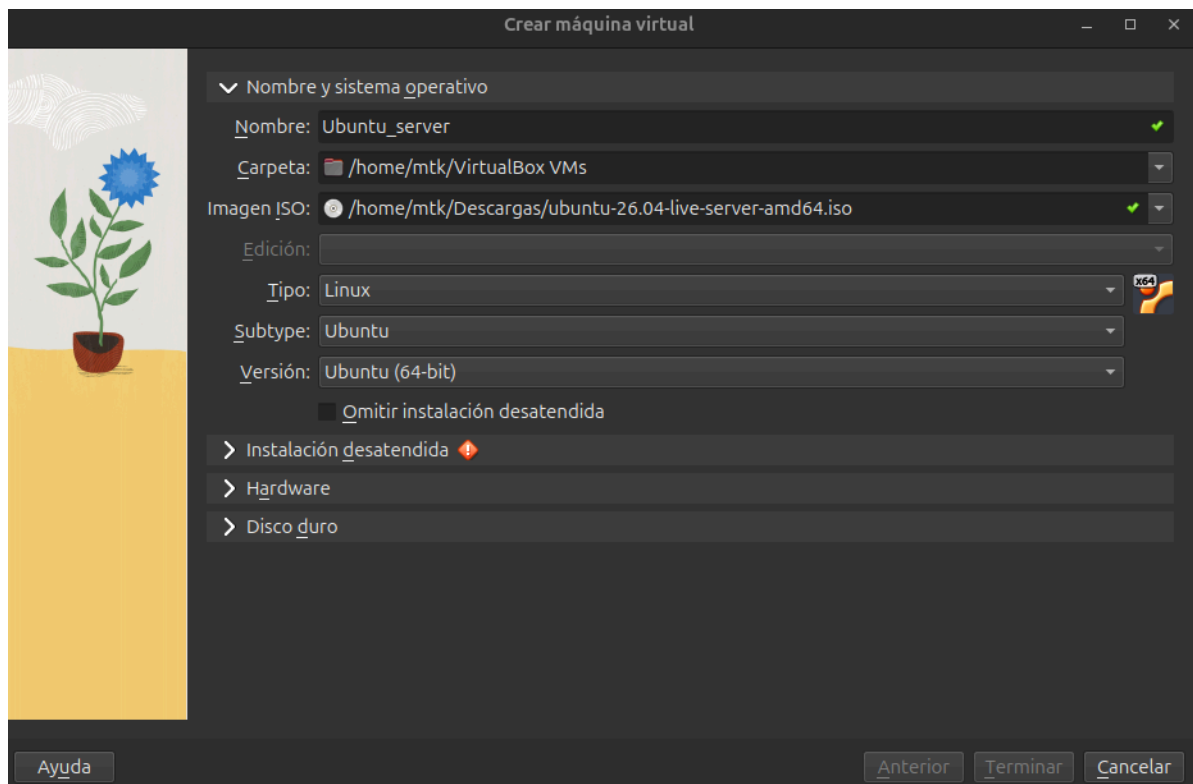


4.2 Instalación del servidor GoPhish

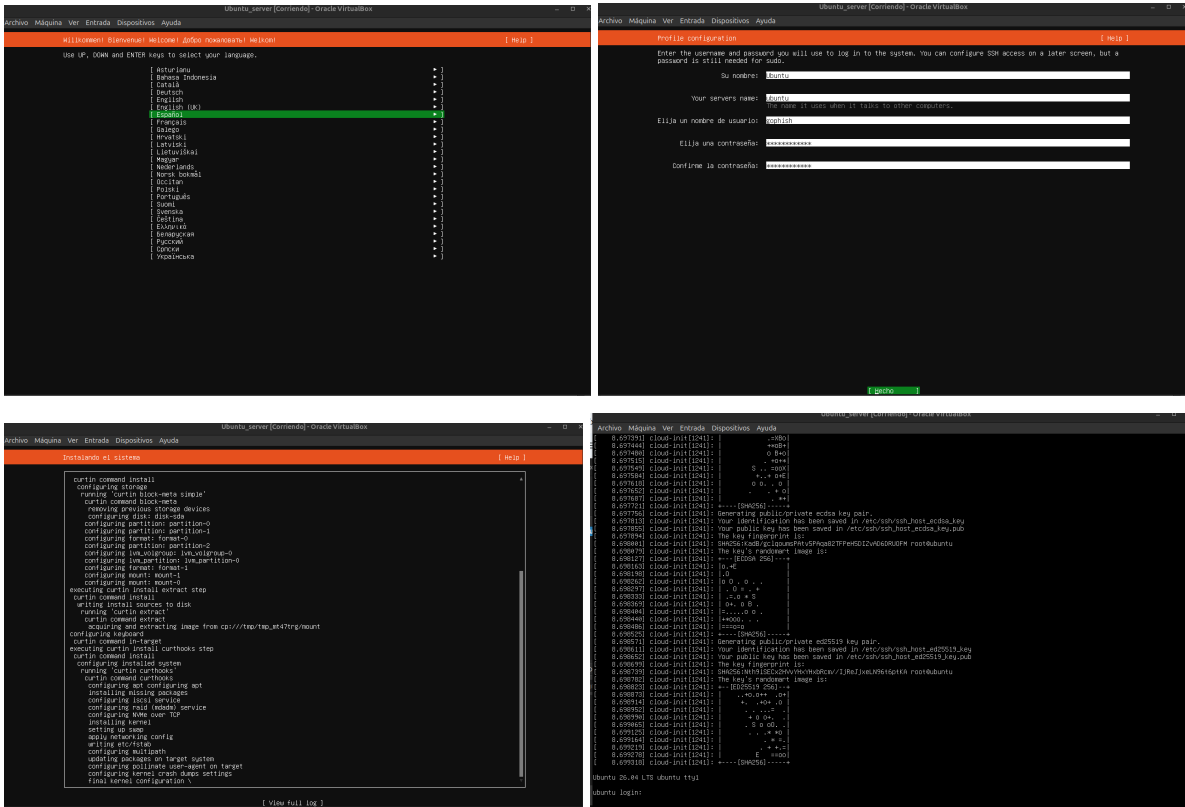
La primera máquina virtual del laboratorio será el servidor donde más adelante se instalará GoPhish. Para ello se ha utilizado Ubuntu Server, una distribución Linux muy utilizada en servidores por su estabilidad y por consumir pocos recursos.



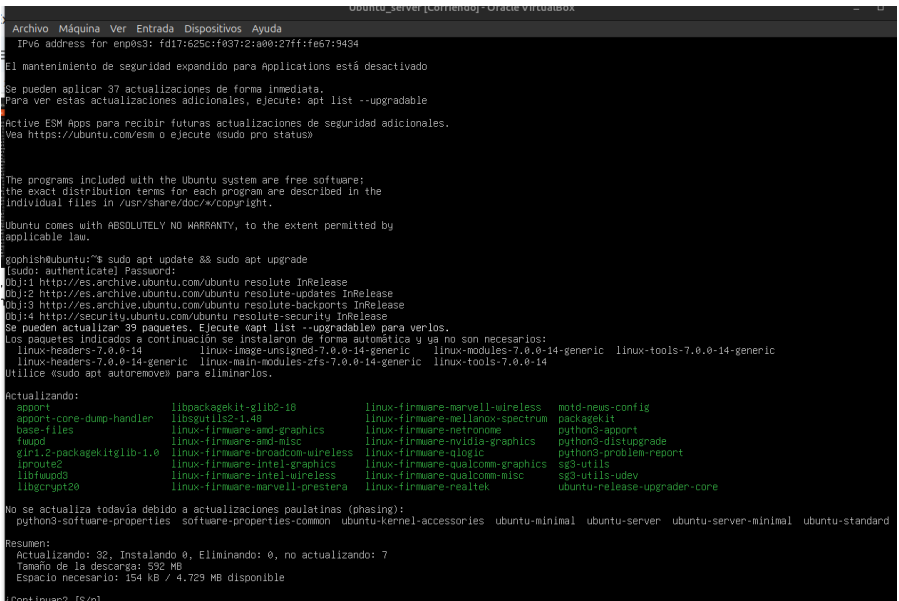
La máquina virtual se creó en VirtualBox con 2 GB de memoria RAM, 2 procesadores y un disco duro virtual de 20 GB. Después, se cargó la imagen ISO de Ubuntu Server y comenzó la instalación.



Durante el proceso se configuraron los datos básicos del sistema, como el idioma, la distribución del teclado, el nombre del equipo, el usuario administrador y el almacenamiento. Una vez terminada la instalación, se comprobó que el servidor arrancaba correctamente y que era posible iniciar sesión.

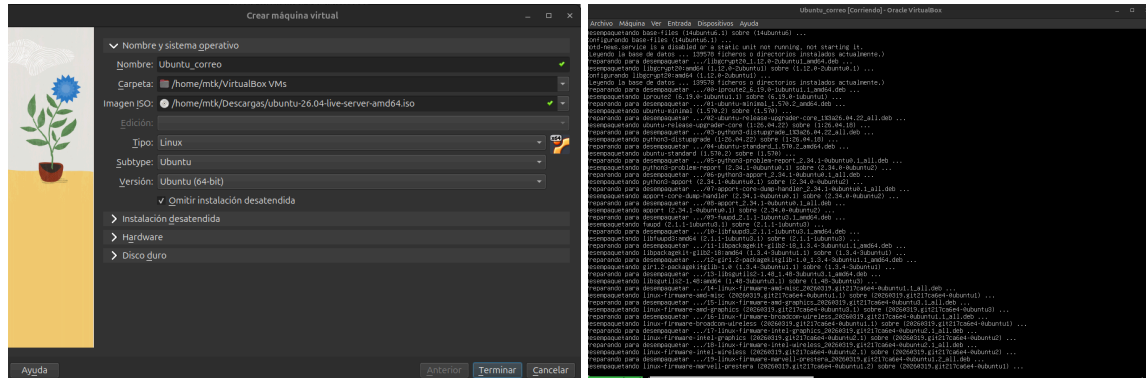


Durante la instalación también se habilitó el servicio OpenSSH, lo que permite la administración remota del servidor desde otros equipos del laboratorio. Por último, antes de instalar GoPhish, se actualizó el sistema operativo para disponer de los paquetes más recientes y trabajar sobre un entorno actualizado.



4.3 Instalación de Ubuntu Server (Servidor de correo)

Después de crear el servidor de GoPhish, se creó una segunda máquina virtual que se utilizará como servidor de correo. Para ello también se eligió Ubuntu Server, ya que es una distribución estable, ligera y adecuada para instalar servicios como Postfix y Dovecot. La máquina virtual se configuró con 2 GB de memoria RAM, 2 procesadores y un disco duro virtual de 20 GB. A continuación, se montó la imagen ISO de Ubuntu Server y comenzó la instalación.



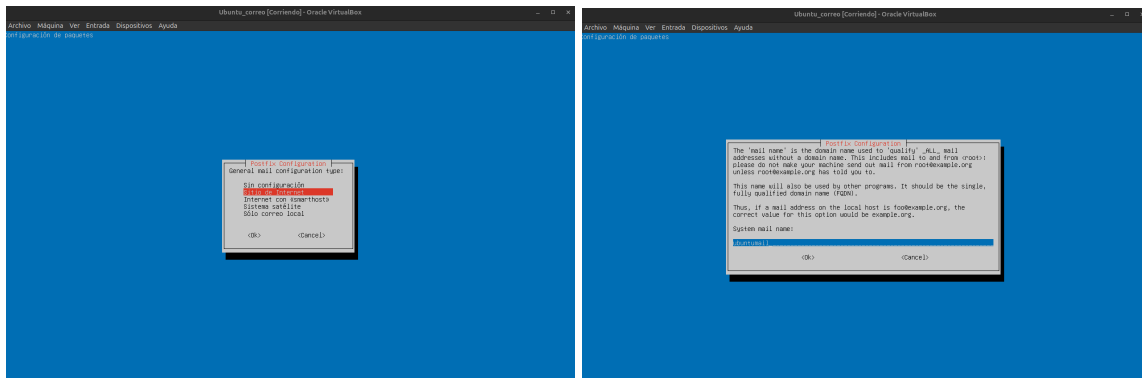
4.3.1 Instalación de Postfix

Una vez preparado el servidor de correo, se instaló Postfix, el servicio que se encargará de enviar y recibir los correos electrónicos utilizados en las simulaciones de phishing.

Se eligió Postfix por ser uno de los servidores de correo más utilizados en Linux, además de destacar por su estabilidad y por ser sencillo de configurar.



Como dominio se escogió, empresa.lab



Una vez instalado Postfix, fue necesario configurarlo para que pudiera enviar y recibir correos correctamente dentro del laboratorio. Para ello se modificó su archivo principal de configuración, `/etc/postfix/main.cf`, donde se definen los parámetros básicos del servidor.

Para editar este archivo se utilizó el editor de texto nano con el siguiente comando:
`sudo nano /etc/postfix/main.cf`

```

Ubuntu_correo [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
GNU nano 8.7.1 /etc/postfix/main.cf
# See /usr/share/postfix/main.cf.dist for a commented, more complete version

# See http://www.postfix.org/COMPATIBILITY_README.html
compatibility_level = 3.9

# Which domain that locally-originated mail appears to come from.
# Debian policy suggests to read this value from /etc/mailname.
#XX needs a review in postinst&config
myorigin = /etc/mailname
myorigin = $mydomain
myorigin = /etc/mailname

# Text that follows the 220 code in the SMTP server's greeting banner.
# You MUST specify $myhostname at the start due to an RFC requirement.
smtpd_banner = $myhostname ESMTP $mail_name (Ubuntu)

# IP protocols to use: ipv4, ipv6, or all
# (set this explicitly so 'post-install upgrade-configuration' won't complain)
inet_protocols = all

# List of "trusted" SMTP clients (maptype:mapname allowed) that have more
# privileges than "strangers". If mynetworks is not specified (the default),
# mynetworks_style is used to compute its value.
mynetworks_style = class
mynetworks_style = subnet
mynetworks_style = host

mynetworks = 127.0.0.0/8 [::ffff:127.0.0.0]/104 [::1]/128

# Uncomment the next line to generate "delayed mail" warnings
delay_warning_time = 4h

# List of domains (maptype:mapname allowed) that this machine considers
# itself the final destination for.
mydestination = $myhostname, empresa.lab, ubuntu1mail, localhost.localdomain, localhost

# Maximum size of a user mailbox
mailbox_size_limit = 0

# Optional external command to use instead of mailbox delivery. If set,
# you must set up an alias to forward root mail to a real user.
#mailbox_command = /usr/bin/procmail
#mailbox_command = /usr/bin/procmail -a "$EXTENSION"
#mailbox_command =

# List of alias maps to use to lookup local addresses.
[ Read 88 lines ]

```

Se cambió el nombre del servidor modificando el parámetro `myhostname` para que pasara a ser `mail.empresa.lab`, asignándole un nombre de dominio completo (FQDN).

```

# SMTP client TLS security level: none|may|encrypt|...
smtpd_tls_security_level = may
# SMTP Client TLS session cache
smtpd_tls_session_cache_database = btree:${data_directory}/smtpd_scache
smtpd_relay_restrictions = permit_mynetworks permit_sasl_authenticated defer_unauth_destination
myhostname = mail.empresa.lab
mydomain = empresa.lab
inet_interfaces = all

```

Se añadió el parámetro mydomain con el valor empresa.lab, que será el dominio utilizado en el laboratorio y por último, se modificó el parámetro myorigin para que los correos enviados utilizaran el dominio empresa.lab como dirección de origen.

```
# Which domain that locally-originated mail appears to come from.
# Debian policy suggests to read this value from /etc/mailname.
#XX needs a review in postinst&config
myorigin = /etc/mailname
myorigin = $mydomain
myorigin = /etc/$mydomain
# Text that follows the 220 code in the SMTP service greeting banner
```

Después se modificó el parámetro mydestination para indicar los dominios que el servidor debe considerar como locales, utilizando una configuración más sencilla basada en las variables de Postfix.

```
# List of domains (maptype:mapname allowed) that this machine considers
# itself the final destination for.
mydestination = $myhostname, localhost.localdomain, localhost, $mydomain
# Maximum size of a user mailbox
mailbox_size_limit = 0
```

El parámetro inet_interfaces se dejó con el valor all, permitiendo que el servidor escuche las conexiones en todas sus interfaces de red.

Por último, el parámetro mynetworks se mantuvo con la configuración predeterminada. Más adelante, cuando se asignen las direcciones IP estáticas del laboratorio, se añadirá la red interna para que los equipos puedan utilizar el servidor de correo.

Una vez realizados los cambios, se guardó el archivo de configuración y se comprobó que no existieran errores con el siguiente comando:

sudo postfix check

```
# List of alias maps to use to lookup local addresses.

user1@ubuntu1:~$ sudo postfix check
postfix/postfix-script: warning: not owned by root: /var/spool/postfix/etc/resolv.conf
user1@ubuntu1:~$
```

Se reinició el servicio para aplicar la nueva configuración y se verificó que Postfix se hubiera iniciado correctamente comprobando el estado del servicio

```
user1@ubuntu1:~$ sudo systemctl restart postfix
user1@ubuntu1:~$ sudo systemctl status postfix
● postfix.service - Postfix Mail Transport Agent (main/default instance)
   Loaded: loaded (/usr/lib/systemd/system/postfix.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-07-24 12:11:58 UTC; 19s ago
   Invocation: ecf2805d54ef46788254df91be3b328b
   Docs: man:postfix(1)
   Process: 13593 ExecStartPre=postfix check (code=exited, status=0/SUCCESS)
   Process: 13691 ExecStart=postfix debian-systemd-start (code=exited, status=0/SUCCESS)
```

4.3.2 Instalación y configuración de Dovecot

Una vez configurado Postfix, el siguiente paso fue instalar Dovecot, que será el encargado de gestionar el acceso a los buzones de correo. En este proyecto se utilizará el protocolo IMAP, ya que permite que los mensajes se almacenen en el servidor y puedan consultarse desde el cliente de correo de Windows 10.

Para instalar Dovecot se utilizó el siguiente comando:

```
sudo apt install dovecot-imapd
```

```
user1@ubuntu:~$ sudo apt install dovecot-imapd
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios:
  linux-headers-7.0.0-14      linux-image-unsigned-7.0.0-14-generic  linux-modules-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-headers-7.0.0-14-generic  linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14      pollinate
Utilice «sudo apt autoremove» para eliminarlos.

Instalando:
  dovecot-imapd

Instalando dependencias:
  dovecot-core dovecot-sieve libexttextcat-2.0-0 libexttextcat-data liblua5.4-0 libpcre2-32-0

Paquetes sugeridos:
  dovecot-flatcurve dovecot-ldap dovecot-managesieved dovecot-pgsql dovecot-solr dovecot-submissiond
  dovecot-gssapi dovecot-lmtpd dovecot-mysql dovecot-pop3d dovecot-sqlite

Resumen:
  Actualizando: 0, Instalando 7, Eliminando: 0, no actualizando: 2
  Tamaño de la descarga: 4.260 kB
  Espacio necesario: 15,6 MB / 4.722 MB disponible

¿Continuar? [S/n]
```

Una vez finalizada la instalación, se comprobó que el servicio se había iniciado correctamente.

```
user1@ubuntu:~$ sudo systemctl status dovecot
* dovecot.service - Dovecot IMAP/POP3 email server
   Loaded: loaded (/usr/lib/systemd/system/dovecot.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-07-24 12:23:22 UTC; 48s ago
   Invocation: 32a06062b92b4d37aa31b38c3e88b060
   Docs: man:dovecot(1)
        https://doc.dovecot.org/
   Main PID: 16511 (dovecot)
   Status: "v2.4.2 (0962ed2104) running"
   Tasks: 4 (limit: 1885)
   Memory: 7.4M (peak: 9.5M)
   CPU: 66ms
   CGroup: /system.slice/dovecot.service
           └─16511 /usr/sbin/dovecot -F
             └─16512 dovecot/anvil
               └─16513 dovecot/log
                 └─16514 dovecot/config
```

Dovecot se configuró para utilizar el formato Maildir como sistema de almacenamiento del correo. Con este formato, cada mensaje se guarda en un archivo independiente, lo que hace que el servicio sea más fiable y evita problemas cuando varios procesos acceden al mismo buzón al mismo tiempo.

```
Ubuntu_correo [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 8.7.1 /etc/dovecot/conf.d/10-mail.conf *
mailboxes that are managed by sysadmin. If you create any shared or public
namespaces you'll typically want to enable ACL plugin also, otherwise all
users can access all the shared mailboxes, assuming they have permissions
on filesystem level to do so.
namespace inbox {
# Namespace type: private, shared or public
#type = private

# Hierarchy separator to use. You should use the same separator for all
# namespaces or some clients get confused. '/' is usually a good one.
# The default however depends on the underlying mail storage format.
#separator =

# Prefix required to access this namespace. This needs to be different for
# all namespaces. For example "Public/".
#prefix =

# Physical location of the mailbox. This is in same format as
mail_location =maildir:~/Maildir
# mail_driver =
# mail_path =
#
# There can be only one INBOX, and this setting defines which namespace
# has it.
inbox = yes

# If namespace is hidden, it's not advertised to clients via NAMESPACE
# extension. You'll most likely also want to set list=no. This is mostly
# useful when converting from another server with different namespaces which
# you want to deprecate but still keep working. For example you can create
# hidden namespaces with prefixes "~/mail/", "~/xu/mail/" and "mail/".
#hidden = no

# Show the mailboxes under this namespace with LIST command. This makes the
# namespace visible for clients that don't support NAMESPACE extension.
# "children" value lists child mailboxes, but hides the namespace prefix.
#list = yes

# Namespace handles its own subscriptions. If set to "no", the parent
# namespace handles them (empty prefix should always have this as "yes")
#subscriptions = yes

# See 15-mailboxes.conf for definitions of special mailboxes.
```

También se configuró la autenticación de Dovecot para permitir el acceso mediante los métodos PLAIN y LOGIN, ya que son compatibles con la mayoría de los clientes de correo. Al tratarse de un laboratorio aislado, se permitió la autenticación en texto plano para facilitar las pruebas realizadas durante el proyecto.

```
#!/include auth-master.conf.ext
#!/include auth-oauth2.conf.ext

#!/include auth-system.conf.ext
#!/include auth-sql.conf.ext
#!/include auth-ldap.conf.ext
#!/include auth-passwdfile.conf.ext
#!/include auth-static.conf.ext
disable_plaintext_auth = no
```

Durante la configuración de Dovecot surgieron varias incidencias relacionadas con cambios introducidos en la versión 2.4.2 del servidor de correo. La mayoría de la documentación consultada estaba basada en versiones anteriores (2.3.x), por lo que algunos parámetros utilizados inicialmente ya no eran compatibles.

La primera incidencia apareció al reiniciar el servicio tras modificar el archivo 10-auth.conf. Dovecot mostraba el siguiente error indicando que el parámetro disable_plaintext_auth ya no era reconocido.

```
* dovecot.service - Dovecot IMAP/POP3 email server
   Loaded: loaded (/usr/lib/systemd/system/dovecot.service; enabled; preset: enabled)
   Active: failed (Result: exit-code) since Fri 2026-07-24 12:44:25 UTC; 5min 19s ago
   Duration: 21min 3.426s
   Invocation: 888fe40676084711b46e35d6cc79f122
   Docs: man:dovecot(1)
        https://doc.dovecot.org/
   Process: 16722 ExecStart=/usr/sbin/dovecot -F (code=exited, status=89)
   Main PID: 16722 (code=exited, status=89)
   Mem peak: 2.8M
   CPU: 24ms

jul 24 12:44:25 ubuntu@mail systemd[1]: Starting dovecot.service - Dovecot IMAP/POP3 email server...
jul 24 12:44:25 ubuntu@mail dovecot[16722]: dovecot: Fatal: Error in configuration file /etc/dovecot/conf.d/10-auth.conf line 121: disable_plaintext_auth: Unk
jul 24 12:44:25 ubuntu@mail systemd[1]: dovecot.service: Main process exited, code=exited, status=89/n/a
jul 24 12:44:25 ubuntu@mail systemd[1]: dovecot.service: Failed with result 'exit-code'.
jul 24 12:44:25 ubuntu@mail systemd[1]: Failed to start dovecot.service - Dovecot IMAP/POP3 email server.
```

```
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
user1@ubuntu:~$ sudo doveconf -n
# 2.4.2 (0962ed2104): /etc/dovecot/dovecot.conf
# Pigeonhole version 2.4.2 (767418c3)
doveconf: Fatal: Error in configuration file /etc/dovecot/conf.d/10-auth.conf line 121: disable_plaintext_auth: Unknown setting: disable_plaintext_auth
user1@ubuntu:~$
```

La configuración se adaptó utilizando los nuevos parámetros `mail_driver` y `mail_path`, resolviendo la incompatibilidad y se creó la estructura Maildir para el usuario del sistema.

```
user1@ubuntu:~$ sudo systemctl restart dovecot
user1@ubuntu:~$ sudo systemctl status dovecot
● dovecot.service - Dovecot IMAP/POP3 email server
   Loaded: loaded (/usr/lib/systemd/system/dovecot.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-07-24 15:01:05 UTC; 17s ago
  Invocation: 9233e1ba6a71448889adb54602c418b3
     Docs: man:dovecot(1)
           https://doc.dovecot.org/
   Main PID: 17041 (dovecot)
   Status: "v2.4.2 (0962ed2104) running"
     Tasks: 4 (limit: 1885)
    Memory: 7.5M (peak: 9.5M)
       CPU: 54ms
   CGroup: /system.slice/dovecot.service
           └─17041 /usr/sbin/dovecot -F
             └─17042 dovecot/anvil
               └─17043 dovecot/log
                 └─17044 dovecot/config

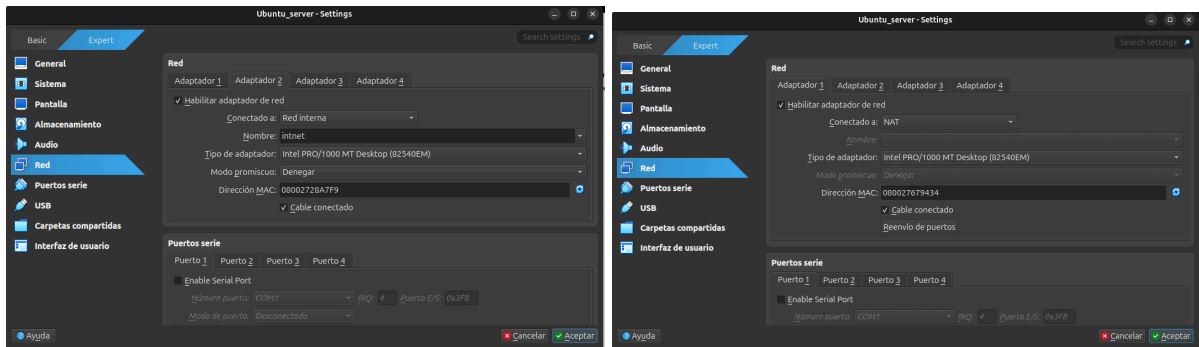
Jul 24 15:01:04 ubuntu: systemd[1]: Starting dovecot.service - Dovecot IMAP/POP3 email server...
Jul 24 15:01:05 ubuntu: dovecot[17041]: master: Dovecot v2.4.2 (0962ed2104) starting up for imap (core dumps disabled)
Jul 24 15:01:05 ubuntu: systemd[1]: Started dovecot.service - Dovecot IMAP/POP3 email server.
user1@ubuntu:~$ sudo ss -tlnp | grep ":143"
LISTEN 0      100        0.0.0.0:*        users:((("dovecot",pid=17041,fd=33))
LISTEN 0      100        [::]:143        [::]:*        users:((("dovecot",pid=17041,fd=34))
user1@ubuntu:~$
```

4.4 Configuración de la red

Una vez instalados los servidores, se configuró la red para que las máquinas virtuales pudieran comunicarse entre sí. Para ello, cada máquina se configuró con **dos adaptadores de red**: uno para acceder a Internet y administrar el sistema, y otro dedicado únicamente a la comunicación interna del laboratorio. De esta forma, el entorno permanece aislado de la red doméstica y las pruebas de phishing pueden realizarse de forma segura.

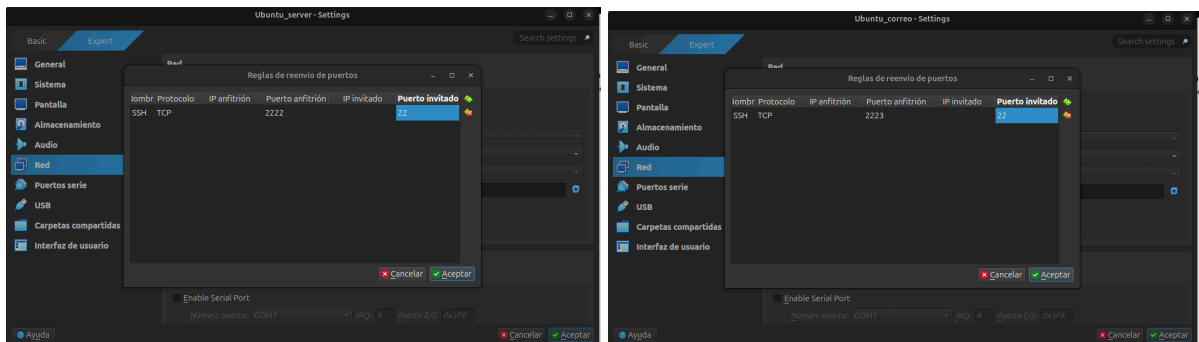
4.4.1 Configuración de las interfaces de red.

Cada una de las máquinas virtuales se configuró con dos adaptadores de red en VirtualBox. El primero se configuró en modo NAT, lo que permite acceder a Internet para instalar paquetes, descargar actualizaciones y realizar tareas de administración. El segundo se configuró en modo Red interna y se utiliza para la comunicación entre las tres máquinas del laboratorio: el servidor de GoPhish, el servidor de correo y el equipo con Windows 10. Todo el tráfico de las pruebas de phishing circula por esta red privada, manteniendo el laboratorio aislado de la red doméstica.



4.4.2 Configuración del acceso remoto mediante SSH

Para poder administrar los servidores Linux de forma más cómoda, se configuró una redirección de puertos (Port Forwarding) en el adaptador NAT de cada máquina virtual. Gracias a esta configuración, es posible conectarse por SSH desde el equipo anfitrión utilizando un puerto diferente para cada servidor, sin necesidad de que las máquinas virtuales sean visibles en la red local.



```
mtk@mtk-IdeaPad-3-15ITL6:~$ ssh gophish@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:Nth9lSECx2HVvYHxYHxbRcm//IjReJjxeLN96t6ptKA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
gophish@localhost's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-28-generic x86_64)
```

```
* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of sáb 25 jul 2026 09:57:09 UTC

System load:        0.03
Usage of /:          50.0% of 9.75GB
Memory usage:       15%
Swap usage:          0%
Processes:          120
Users logged in:     0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe67:9434
```

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 0 actualizaciones de forma inmediata.

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea <https://ubuntu.com/esm> o ejecute «sudo pro status»

```
gophish@ubuntu:~$
```

```
user1@ubuntumail:~$ whoami
user1
user1@ubuntumail:~$ pwd
/home/user1
user1@ubuntumail:~$
```

4.4.3 Configuración de direcciones IP estáticas

Una vez configurada la red, se asignó una dirección IP estática a cada una de las máquinas del laboratorio. De esta forma, cada equipo mantiene siempre la misma dirección IP, lo que facilita la comunicación entre los diferentes servicios.

En los servidores Ubuntu la configuración se realizó con Netplan, asignando las siguientes IPs:

Máquina	Dirección IP
Servidor GoPhish	192.168.100.10
Servidor de correo	192.168.100.20

```
GNU nano 3.7.1 /etc/netplan/00-installer-config.yaml
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp8s1:
      dhcp4: true
      dhcp6: true
      macchi:
        macaddress: 08:00:27:b4:fa:f2
      set-name: enp8s3
    enp8s2:
      dhcp4: false
      dhcp6: false
      addresses:
        - 192.168.100.20/24
  version: 2
```

```
[sudo: authenticate] Password:
user1@ubuntumail:~$ sudo netplan try
Do you want to keep these settings?

Press ENTER before the timeout to accept the new configuration

Changes will revert in 119 seconds
Configuration accepted.
user1@ubuntumail:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp8s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:b4:fa:f2 brd ff:ff:ff:ff:ff:ff
    altname enx80027b4faf2
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp8s3
        valid_lft 86394sec preferred_lft 86394sec
    inet6 fd1:435c:fb37:2:a00:27ff:fe84:fa2/64 scope global dynamic mngtnpaddr noprefixroute
        valid_lft 86395sec preferred_lft 14395sec
    inet6 fe80::a00:27ff:fe84:fa2/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: enp8s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:b0:5c:ac brd ff:ff:ff:ff:ff:ff
    altname enx80027b05cac
    inet 192.168.100.20/24 brd 192.168.100.255 scope global enp8s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe80:5cac/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
user1@ubuntumail:~$
```

4.4.4 Verificación de la conectividad

Una vez configurada la red, se realizó la prueba para comprobar que todo funcionaba correctamente mediante ping.

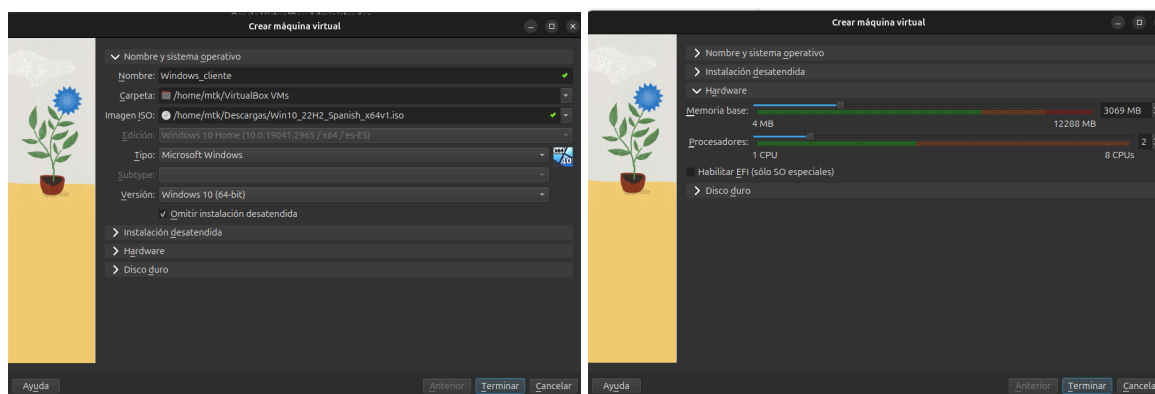
```
user1@ubuntumail:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=0.977 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=0.488 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=0.536 ms
^C
--- 192.168.100.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2048ms
rtt min/avg/max/mdev = 0.488/0.667/0.977/0.220 ms
user1@ubuntumail:~$
```

```
gophish@ubuntu:~$ ping 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.
64 bytes from 192.168.100.20: icmp_seq=1 ttl=64 time=0.462 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=64 time=0.488 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=64 time=0.480 ms
^C
--- 192.168.100.20 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2081ms
rtt min/avg/max/mdev = 0.462/0.476/0.488/0.010 ms
gophish@ubuntu:~$
```

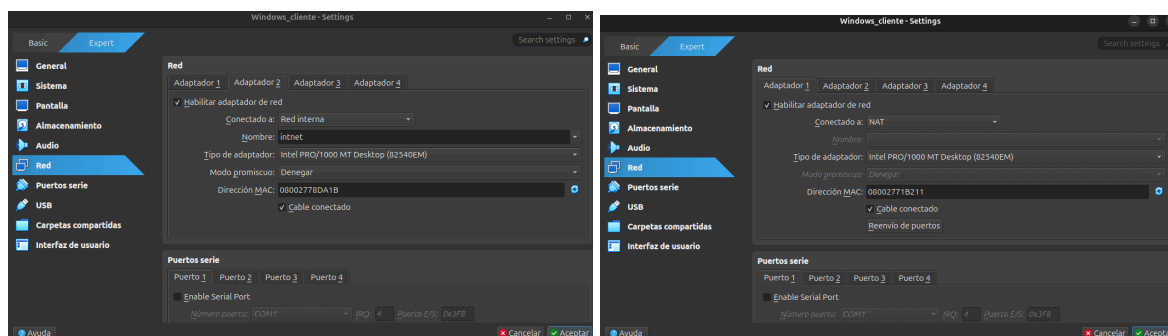
4.5 Instalación de Windows 10 (Equipo Cliente)

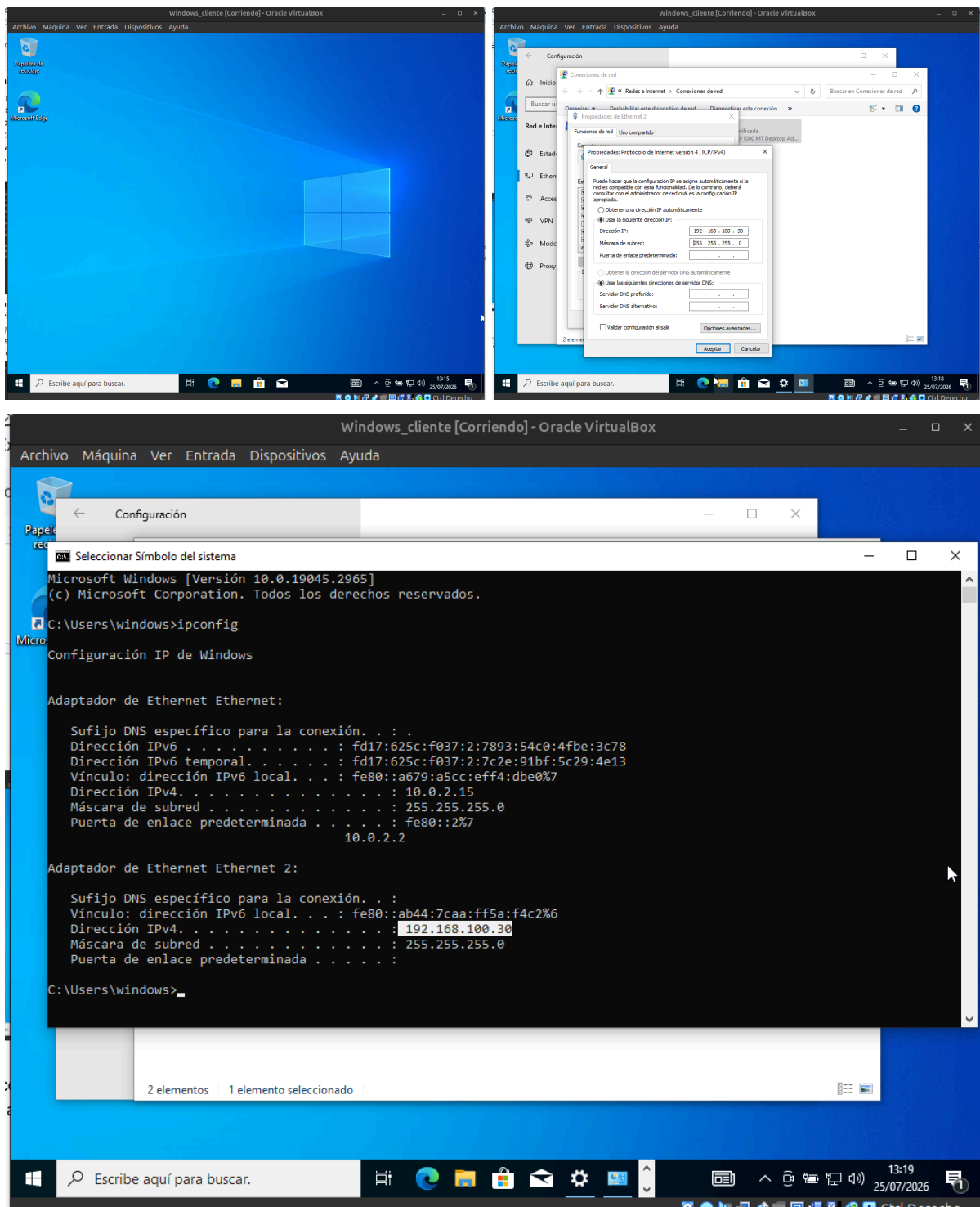
La tercera máquina virtual del laboratorio será el equipo cliente desde el que se realizarán las pruebas de las campañas de phishing. Para ello se utilizó Windows 10, ya que es uno de los sistemas operativos más utilizados en empresas y permite simular un entorno de trabajo más realista.

La máquina virtual se creó en VirtualBox con 2 GB de memoria RAM, 2 procesadores y un disco duro virtual de 40 GB. Después, se cargó la imagen ISO de Windows 10.



A continuación, se instalaron las actualizaciones disponibles y se configuró la red. Igual que en los servidores, el primer adaptador se dejó en modo NAT para tener acceso a Internet, mientras que el segundo se configuró en modo Red interna para comunicarse con el servidor de GoPhish y el servidor de correo. Por último, se asignó una dirección IP estática al adaptador de la red interna.





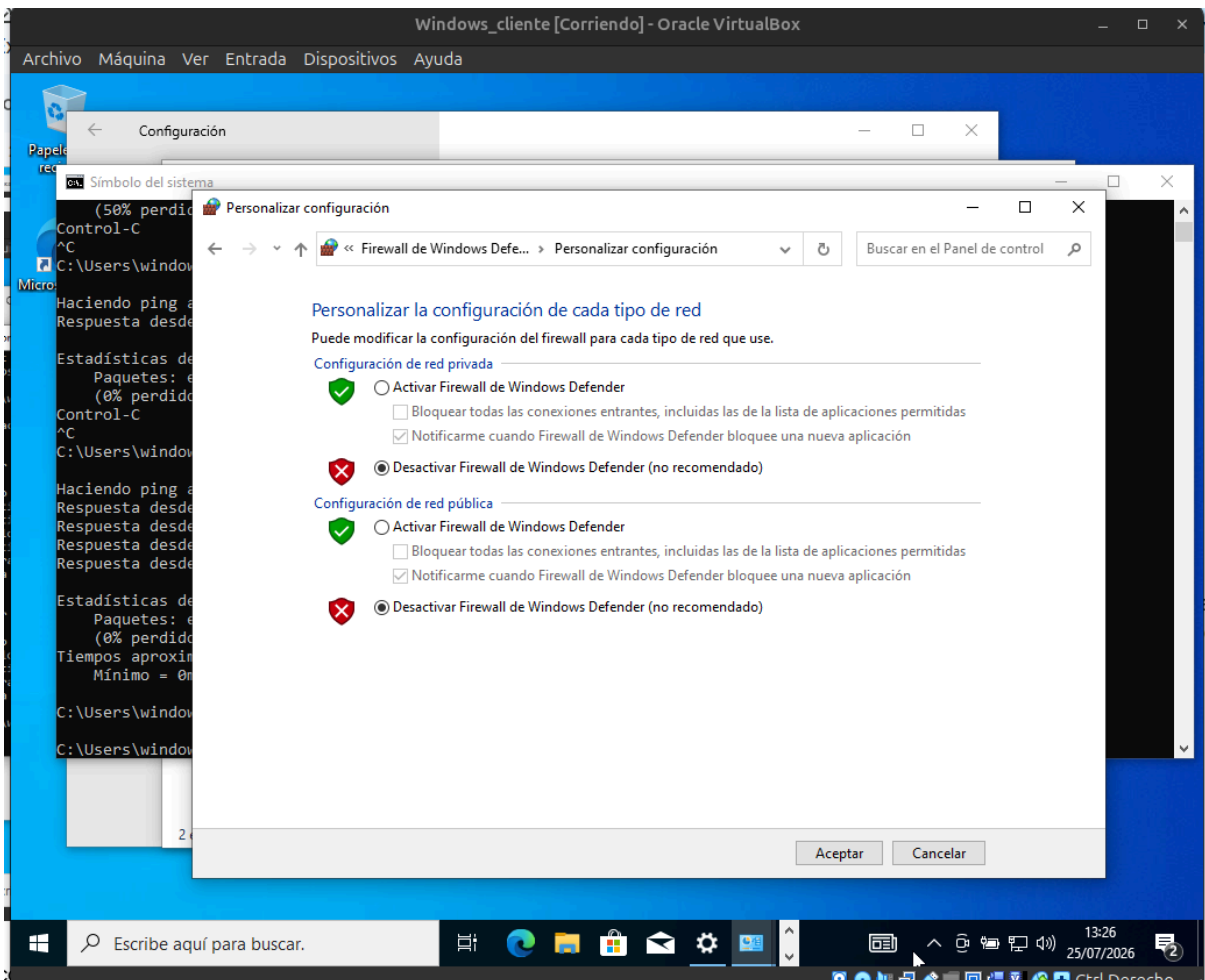
Con esta configuración, el equipo quedó listo para recibir los correos de las campañas de phishing y acceder a las páginas web creadas con GoPhish.

Durante las pruebas de conectividad se observó que Windows 10 no respondía a las pruebas de ping debido a la configuración predeterminada del Firewall de Microsoft Defender. Para verificar la comunicación entre las máquinas se deshabilitó el firewall momentáneamente y luego se volvió a habilitar modificando la regla de entrada correspondiente a las solicitudes de eco (ICMPv4).

```
C:\Users\windows>ping 192.168.100.10

Haciendo ping a 192.168.100.10 con 32 bytes de datos:
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64

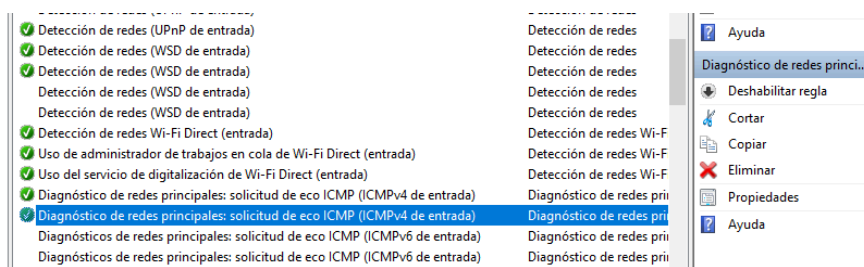
Estadísticas de ping para 192.168.100.10:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
        (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 0ms, Media = 0ms
```



```
C:\Users\windows>ping 192.168.100.10

Haciendo ping a 192.168.100.10 con 32 bytes de datos:
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.100.10: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.100.10:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
        (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 0ms, Media = 0ms
```



4.6 Configuración del cliente de correo

4.6.1 Creación del usuario de correo en el servidor

Una vez instalado el equipo cliente, el siguiente paso fue preparar el servidor de correo para poder realizar las pruebas del laboratorio. Para ello se creó un usuario de correo que actuaría como destinatario de las campañas de phishing. Más adelante, esta cuenta se configuraría en el cliente de correo de Windows 10 para poder recibir los mensajes enviados desde GoPhish.

En este proyecto se creó el usuario empleado, con la dirección de correo empleado@empresa.lab. Esta cuenta se utilizará durante las pruebas para recibir los correos enviados desde GoPhish y comprobar que el servidor de correo funciona. Añadimos empleado a nuestro servidor de correo.

```
user1@ubuntu1:~/home$ sudo adduser empleado
New password:
Retype new password:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para empleado
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
  Nombre completo []: Maria
  Número de habitación []:
  Teléfono del trabajo []:
  Teléfono de casa []:
  Otro []:
Is the information correct? [Y/n] y
user1@ubuntu1:~/home$ ls
empleado user1
user1@ubuntu1:~/home$ _
```

Después de crear el usuario empleado, fue necesario crear la estructura Maildir, ya que Dovecot utiliza este formato para almacenar los correos electrónicos. Para ello se crearon los directorios cur, new y tmp, que son los encargados de organizar los mensajes dentro del buzón y asignamos el propietario.

```
user1@ubuntu1:~/home$ sudo mkdir -p /home/empleado/Maildir/{cur,new,tmp}
user1@ubuntu1:~/home$ sudo chown -R empleado:empleado /home/empleado/Maildir
user1@ubuntu1:~/home$
```

```
user1@ubuntu1:~/home$ sudo do
root@ubuntu1:~/home# ls -la /home/empleado/
total 24
drwxr-xr-- 3 empleado empleado 4096 Jul 26 09:24 .
drwxr-xr-x 4 root      root      4096 Jul 26 09:20 ..
-rw-r--r-- 1 empleado empleado 229 Jul 26 09:20 .bash_logout
-rw-r--r-- 1 empleado empleado 3771 Jul 26 09:20 .bashrc
-rw-r--r-- 1 empleado empleado 807 Jul 26 09:20 .profile
drwxr-xr-x 5 empleado empleado 4096 Jul 26 09:24 Maildir
root@ubuntu1:~/home# ls -la /home/empleado/Maildir/
total 20
drwxr-xr-x 5 empleado empleado 4096 Jul 26 09:24 .
drwxr-xr-- 3 empleado empleado 4096 Jul 26 09:24 ..
drwxr-xr-x 2 empleado empleado 4096 Jul 26 09:24 cur
drwxr-xr-x 2 empleado empleado 4096 Jul 26 09:24 new
drwxr-xr-x 2 empleado empleado 4096 Jul 26 09:24 tmp
root@ubuntu1:~/home#
```

Una vez terminada la configuración, se comprobó que todo funcionaba correctamente. Para ello se verificó que Dovecot utilizaba el formato Maildir para almacenar los correos y que el usuario creado podía acceder a su buzón sin problemas.

```

userdb passwd {
}
namespace inbox {
  mail_driver = maildir
  mail_path = ~/Maildir
  inbox = yes
  mailbox Drafts {
    special_use = "\\Drafts"
  }
}

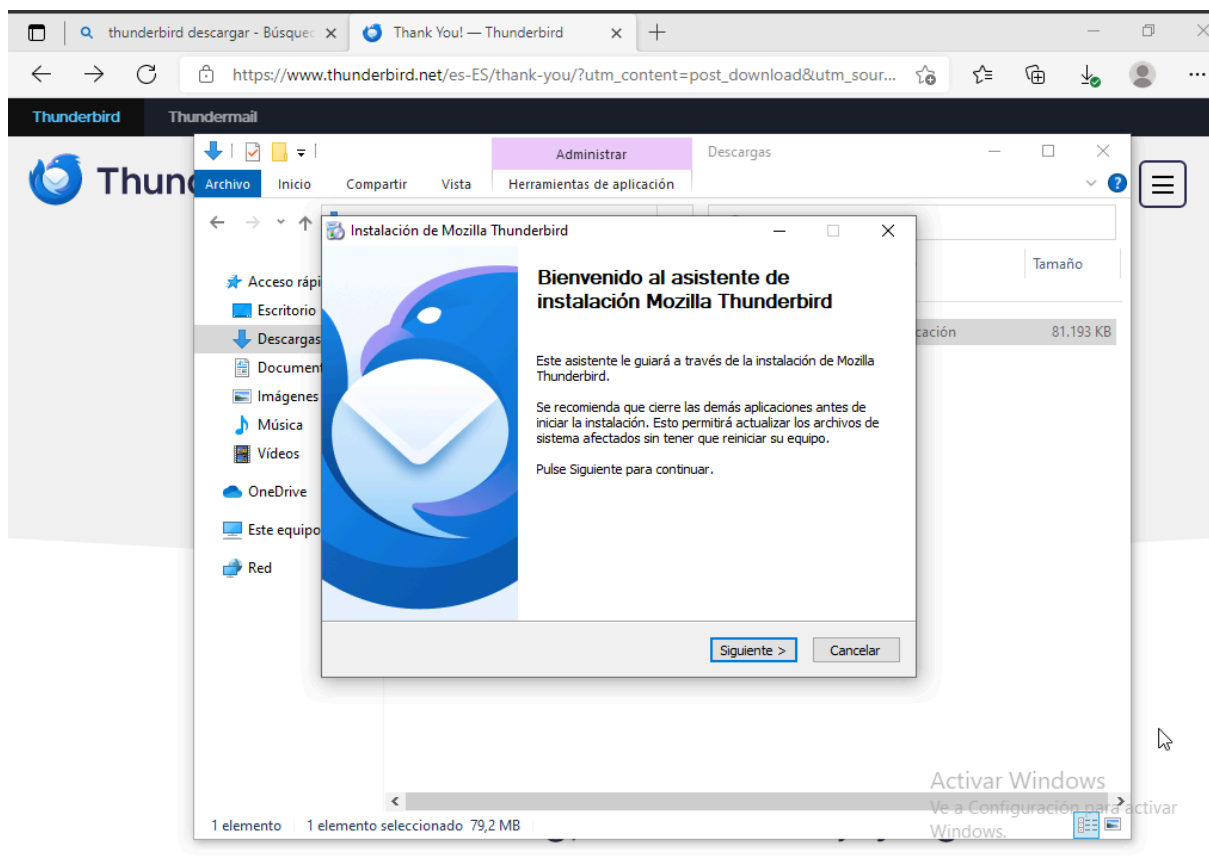
user1@ubuntu:~$ sudo doveadm user empleado
field      value
uid        1001
gid        1001
home       /home/empleado
mail_path
system_groups_user  empleado
user1@ubuntu:~$

```

4.6.2 Instalación de Thunderbird

Una vez creada la cuenta de correo, el siguiente paso fue instalar un cliente de correo electrónico en el equipo con Windows 10 para poder recibir y consultar los mensajes enviados durante las pruebas.

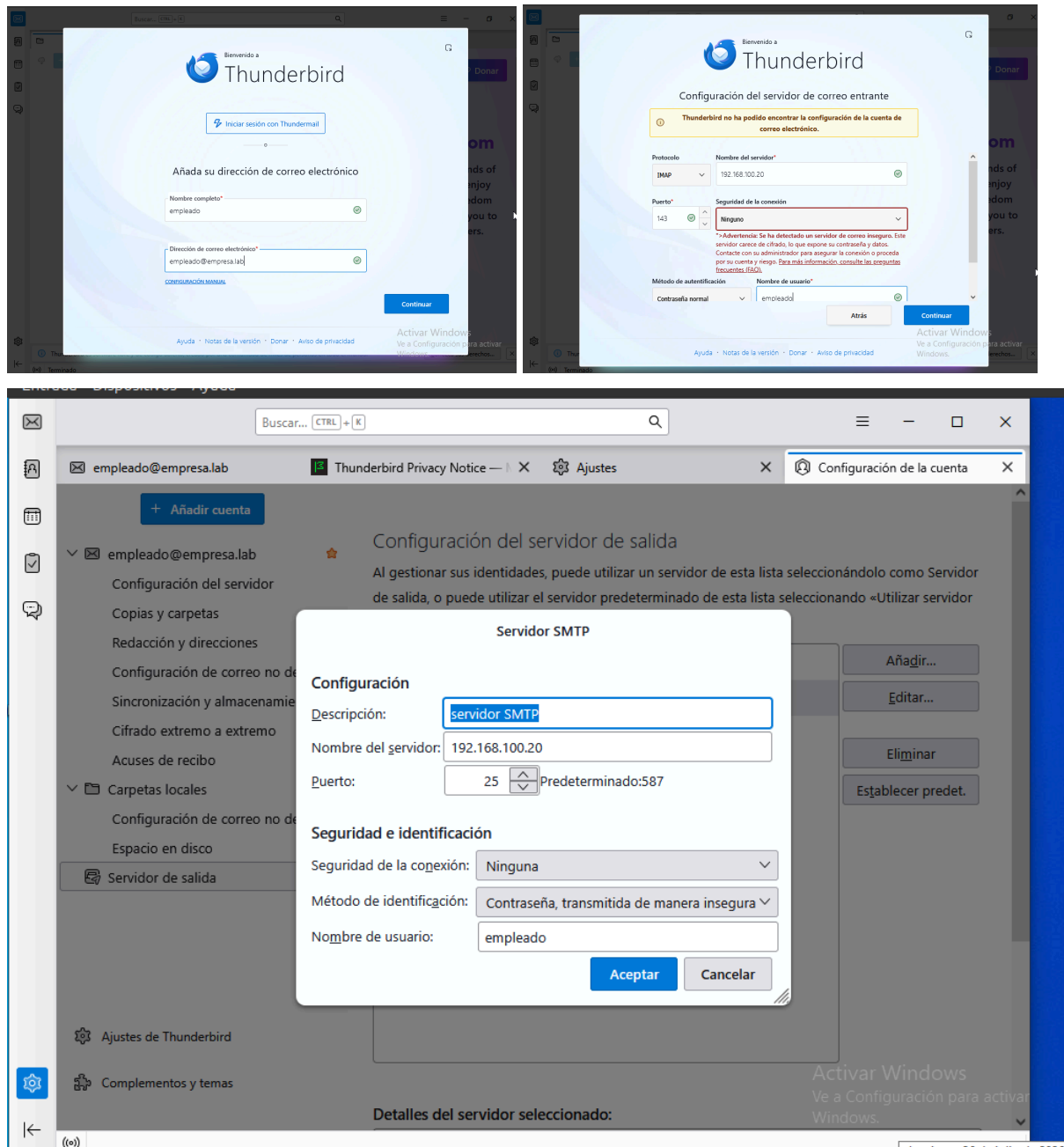
Para ello se eligió Mozilla Thunderbird, ya que es un cliente de correo gratuito, de código abierto y muy utilizado. Además, es compatible con los protocolos SMTP, IMAP y POP3, lo que lo hace una buena opción para el laboratorio.



4.6.3 Configuración de la cuenta

Una vez instalado Thunderbird, se configuró la cuenta de correo empleado@empresa.lab. Como el dominio empresa.lab solo existe dentro del laboratorio, la configuración se realizó manualmente.

Para recibir los correos se configuró el protocolo IMAP, utilizando la dirección 192.168.100.20 y el puerto 143. Para el envío se configuró SMTP, con la misma dirección IP y el puerto 25.

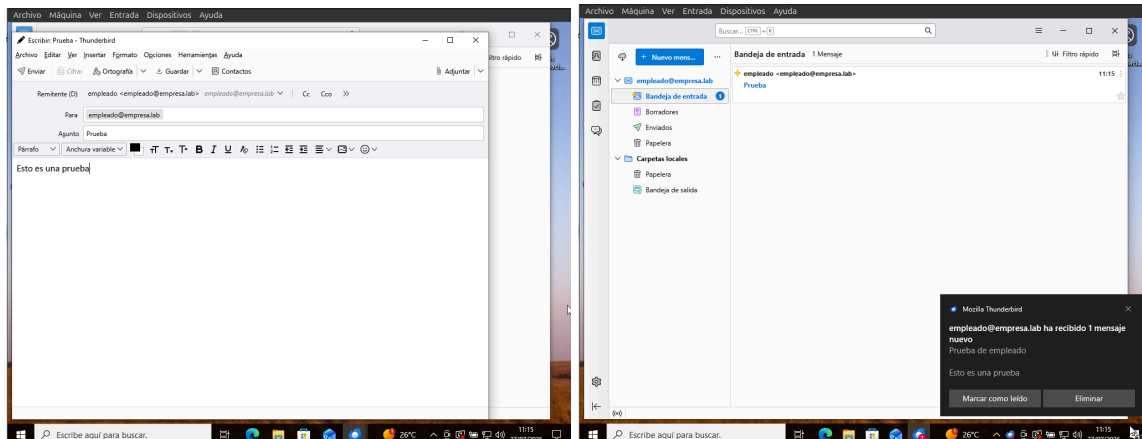


Al tratarse de un entorno de laboratorio, las conexiones se configuraron sin cifrado y sin autenticación SMTP. Una vez finalizada la configuración, se comprobó que Thunderbird podía conectarse correctamente al servidor de correo y acceder a la cuenta creada.

4.6.4 Prueba de funcionamiento

Una vez configurados Thunderbird y el servidor de correo, se realizaron varias pruebas para comprobar que todo funcionaba.

Primero se verificó que Thunderbird podía conectarse al servidor mediante IMAP y acceder al buzón del usuario. Después, se enviaron y recibieron varios correos utilizando la cuenta empleado@empresa.lab, comprobando que los mensajes llegaban.



Por último, se revisaron los registros del servidor para confirmar que los correos habían sido procesados y entregados sin errores.

```
2026-07-27T09:00:42.6767+00:00 ubuntu@mail dovecot: imap-login: Logged in: user=<empleado>, method=PLAIN, rip=192.168.100.30, (ip=192.168.100.20, mpid=5577, session=<7Kk89ZNXCsTAqQe>)
2026-07-27T09:15:29.023245+00:00 ubuntu@mail postfix/smtpd[5589]: connect from unknown[192.168.100.30]
2026-07-27T09:15:29.043876+00:00 ubuntu@mail postfix/smtpd[5589]: 0AADD20204: client=unknown[192.168.100.30]
2026-07-27T09:15:29.050379+00:00 ubuntu@mail postfix/cleanup[5592]: 0AADD20204: message-id=<dfd8533b-da7b-40a7-91f5-4abf09daeba2@empresa.lab>
2026-07-27T09:15:29.054284+00:00 ubuntu@mail postfix/qmgr[5541]: 0AADD20204: from=<empleado@empresa.lab>, size=560, nrcpt=1 (queue active)
2026-07-27T09:15:29.062614+00:00 ubuntu@mail postfix/local[5593]: 0AADD20204: to=<empleado@empresa.lab>, relay=local, delay=0.02, delays=0.02/0/0/0, dsn=2.0.0, status=sent (delivered to maildir)
2026-07-27T09:15:29.062977+00:00 ubuntu@mail postfix/qmgr[5541]: 0AADD20204: removed
2026-07-27T09:15:34.635121+00:00 ubuntu@mail postfix/smtpd[5589]: disconnect from unknown[192.168.100.30] ehlo=1 mail=1 rcpt=1 data=1 quit=1 commands=5
user1@ubuntu:~$
```

Con estas comprobaciones se confirmó que tanto el servidor de correo como Thunderbird funcionaban correctamente y quedaban listos para utilizarse en las campañas de phishing creadas con GoPhish.

4.7 Instalación y configuración de GoPhish

Una vez comprobado que el servidor de correo y Thunderbird funcionaban, el siguiente paso fue instalar GoPhish, la herramienta que se utilizará para realizar las campañas de phishing del laboratorio.

Con GoPhish es posible crear correos electrónicos, diseñar páginas web que imitan sitios reales y gestionar campañas de phishing de forma sencilla. Además, permite conocer si un correo ha sido enviado, si el usuario ha abierto el enlace o si ha introducido sus credenciales durante la simulación.

4.7.1 Instalación de GoPhish

Para instalar GoPhish se utilizó la versión para Linux, instalándola en el servidor Ubuntu preparado para alojar la aplicación.

Primero se actualizó el sistema operativo y, a continuación, se descargó la versión oficial de GoPhish.

<https://github.com/gophish/gophish/releases/latest/download/gophish-v0.12.1-linux-64bit.zip>

```
gophish@ubuntu:/tmp$  
gophish@ubuntu:~$ cd /tmp  
gophish@ubuntu:~/tmp$ wget https://github.com/gophish/gophish/releases/latest/download/gophish-v0.12.1-linux-64bit.zip  
-2026-07-27 09:35:24-- https://github.com/gophish/gophish/releases/latest/download/gophish-v0.12.1-linux-64bit.zip  
Resolving github.com (github.com)... 140.82.121.4  
Connecting to github.com (github.com)[140.82.121.4]:443... connected.  
HTTP request sent, awaiting response... 302 Found  
Location: https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-v0.12.1-linux-64bit.zip [following]  
-2026-07-27 09:35:24-- https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-v0.12.1-linux-64bit.zip  
Reusing existing connection to github.com:443.  
HTTP request sent, awaiting response... 302 Found  
Location: https://release-assets.githubusercontent.com/github-production-release-asset/14508450/d275c2cd-8d50-49e6-a442-fec1d3df01a3?sp=r&sv=2018-11-09&r=b&spr=https&sse=2026-07-27T10%3A12%3A03Z&rscc=attachment%3B+filename%3Dgophish-v0.12.1-linux-64bit.zip&srsc=application%2Foctet-stream&skoid=96c2d410-711-43a1-aedd-ab1947aa7ab08&sktid=398a6654-997b-47e9-b12b-9515b896b4de&st=2026-07-27T10%3A11%3A46Z&tsk=2026-07-27T10%3A12%3A03Z&tsks=b&sv=2018-11-09&si=s=yqVYz52BjE2KfJvQZtIjEnoyqYaT0tDbcbZvgzrqOelUmEK3D&jwt=eY0eXAI0iJKVlQ1JCjhBgCc0iJUzU1NiN39.eYjPc3Mi0iJnaXRodMuY29tIiwiaWwiOiJoicmVsZWZfZS1hc3NldHMud20oAVHdXNmNmNmblbnQyZ9tIiwiaW5ja2VSM5SiImV4cC1MTGt4NTE0bWJoxNgzMjM0ODQ0OT01ClJwYXRoiJoicmVsZWZfZWZfc2V0CHJvZHVjdGlvbSI6bG9iLmNmcmMud2UzG3cySUxQfQfQ.5Z84QYFnxx8VKobbBkendRAJDITkgBpk90akQvsQD_Y&response-content-disposition=attachment%3B%20filename%3Dgophish-v0.12.1-linux-64bit.zip&response-content-type=application%2Foctet-stream [following]  
-2026-07-27 09:35:25-- https://release-assets.githubusercontent.com/github-production-release-asset/14508450/d275c2cd-8d50-49e6-a442-fec1d3df01a3?sp=r&sv=2018-11-09&sr=b&spr=https&sse=2026-07-27T10%3A12%3A03Z&rscc=attachment%3B+filename%3Dgophish-v0.12.1-linux-64bit.zip&srsc=application%2Foctet-stream&skoid=96c2d410-711-43a1-aedd-ab1947aa7ab08&sktid=398a6654-997b-47e9-b12b-9515b896b4de&st=2026-07-27T10%3A11%3A46Z&tsk=2026-07-27T10%3A12%3A03Z&tsks=b&sv=2018-11-09&si=s=yqVYz52BjE2KfJvQZtIjEnoyqYaT0tDbcbZvgzrqOelUmEK3D&jwt=eY0eXAI0iJKVlQ1JCjhBgCc0iJUzU1NiN39.eYjPc3Mi0iJnaXRodMuY29tIiwiaWwiOiJoicmVsZWZfZS1hc3NldHMud20oAVHdXNmNmNmblbnQyZ9tIiwiaW5ja2VSM5SiImV4cC1MTGt4NTE0bWJoxNgzMjM0ODQ0OT01ClJwYXRoiJoicmVsZWZfZWZfc2V0CHJvZHVjdGlvbSI6bG9iLmNmcmMud2UzG3cySUxQfQfQ.5Z84QYFnxx8VKobbBkendRAJDITkgBpk90akQvsQD_Y&response-content-disposition=attachment%3B%20filename%3Dgophish-v0.12.1-linux-64bit.zip&response-content-type=application%2Foctet-stream [following]  
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.111.133, ...  
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)[185.199.110.133]:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 33327057 (32M) [application/octet-stream]  
Saving to: 'gophish-v0.12.1-linux-64bit.zip'  
  
gophish-v0.12.1-linux-64bit.zip          41%[=====]           ] 13.2M/s   3.73MB/s      eta 9s
```

```
gophish@ubuntu:/tmp$ ls -la
total 53592
drwxrwxrwt 14 root root 400 Jul 27 09:37 [redacted]
drwxr-xr-x 20 root root 4096 Jul 24 11:05 [redacted]
drwxrwxrwt 2 root root 40 Jul 27 09:26 [redacted]ICE-unix
drwxrwxrwt 2 root root 40 Jul 27 09:26 [redacted]X11-unix
drwxrwxrwt 2 root root 40 Jul 27 09:26 [redacted]XIM-unix
drwxrwxrwt 2 root root 40 Jul 27 09:26 [redacted]font-unix
-rw-r--r-- 1 root root 1115 Sep 14 2022 LICENSE
-rw-r--r-- 1 root root 3285 Sep 14 2022 README.md
-rw-r--r-- 1 root root 7 Sep 14 2022 VERSION
-rw-r--r-- 1 root root 460 Sep 14 2022 config.json
drwxr-xr-x 4 root root 80 Sep 14 2022 db
-rw-r--r-- 1 root root 21525728 Sep 14 2022 gophish
-rw-rw-r-- 1 gophish gophish 33327057 Sep 14 2022 gophish-v0.12.1-linux-64bit.zip
drwx----- 2 root root 40 Jul 27 09:26 snap-private-tmp
drwxr-xr-x 7 root root 140 Jul 27 09:37 static
drwx----- 3 root root 60 Jul 27 09:26 systemd-private-47198070ec2b48a6be86af4a5c7b3b23-ModemManager.service-kQa1GG
drwx----- 3 root root 60 Jul 27 09:26 systemd-private-47198070ec2b48a6be86af4a5c7b3b23-chrmony.service-gcLfmx
drwx----- 3 root root 60 Jul 27 09:26 systemd-private-47198070ec2b48a6be86af4a5c7b3b23-polkit.service-Q0HJHT
drwx----- 3 root root 60 Jul 27 09:26 systemd-private-47198070ec2b48a6be86af4a5c7b3b23-systemd-logind.service-oJI0NU
drwxr-xr-x 2 root root 40 Sep 14 2022 templates
gophish@ubuntu:/tmp$ sudo chmod +x gophish
```

Después se dieron permisos de ejecución al archivo principal de GoPhish, dejando la aplicación lista para realizar la configuración inicial y ponerla en marcha.

La primera vez que se inició GoPhish, la aplicación creó automáticamente su base de datos interna y los certificados autofirmados necesarios para la interfaz de administración.

También generó toda la estructura necesaria para almacenar las campañas, los usuarios, las plantillas y los resultados.

Por defecto, el panel de administración solo permite conexiones desde el propio servidor.

Como la administración se iba a realizar desde el equipo Windows del laboratorio, fue

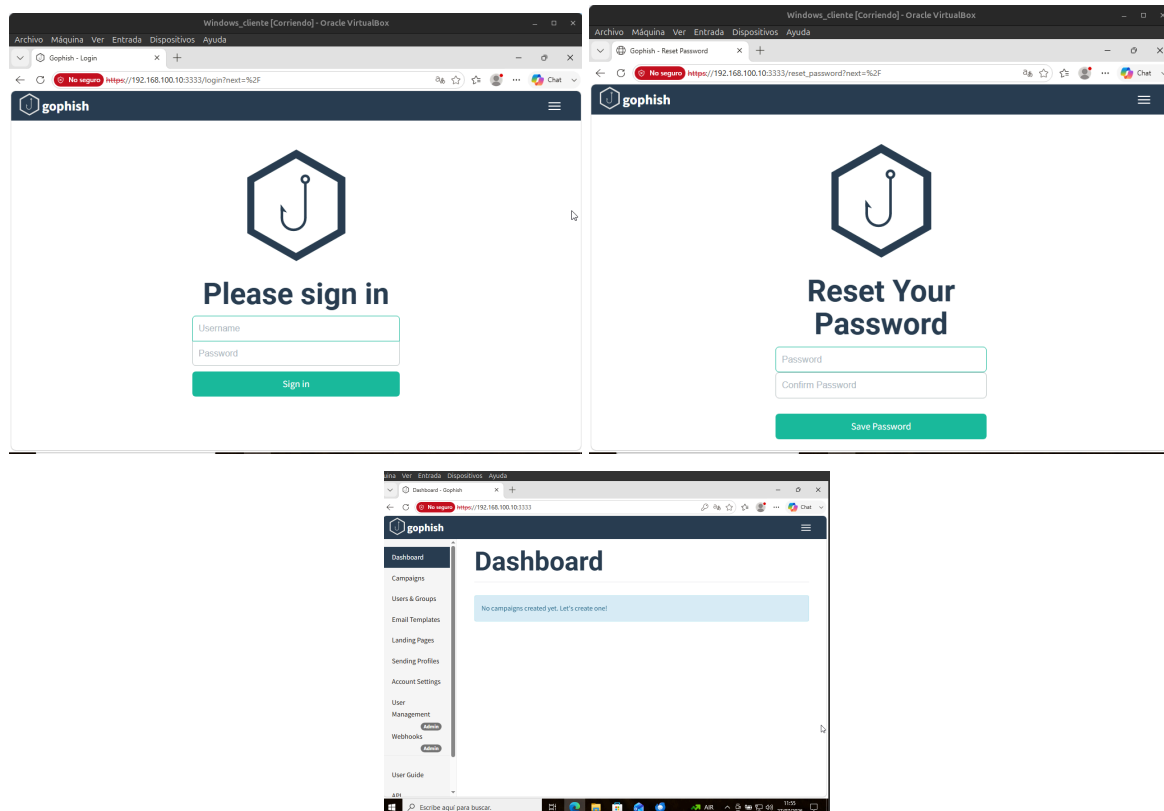
necesario modificar el archivo config.json, cambiando la dirección 127.0.0.1:3333 por 0.0.0.0:3333 para permitir el acceso desde la red interna.

```
gophish@ubuntu:/tmp
GNU nano 8.7.1 config.json *

{
  "admin_server": {
    "listen_url": "0.0.0.0:3333",
    "use_tls": true,
    "cert_path": "gophish_admin.crt",
    "key_path": "gophish_admin.key",
    "trusted_origins": []
  },
  "phish_server": {
    "listen_url": "0.0.0.0:80",
    "use_tls": false,
    "cert_path": "example.crt",
    "key_path": "example.key"
  },
  "db_name": "sqlite3",
  "db_path": "gophish.db",
  "migrations_prefix": "db/db_",
  "contact_address": ""
}
```

Después de guardar los cambios, GoPhish se inició correctamente y quedaron disponibles tanto el panel de administración en el puerto 3333 como el servidor web que alojará las páginas de phishing en el puerto 80.

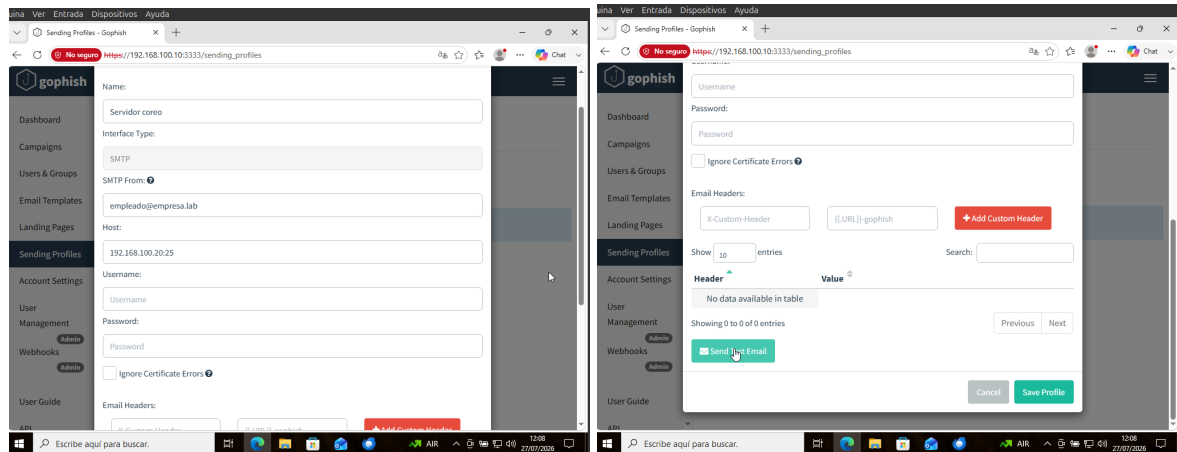
Durante este primer inicio también se generaron las credenciales del usuario administrador. Con ellas fue posible acceder al panel de GoPhish desde el navegador del equipo Windows y comprobar que la instalación se había realizado bien y que la herramienta estaba lista para comenzar su configuración.



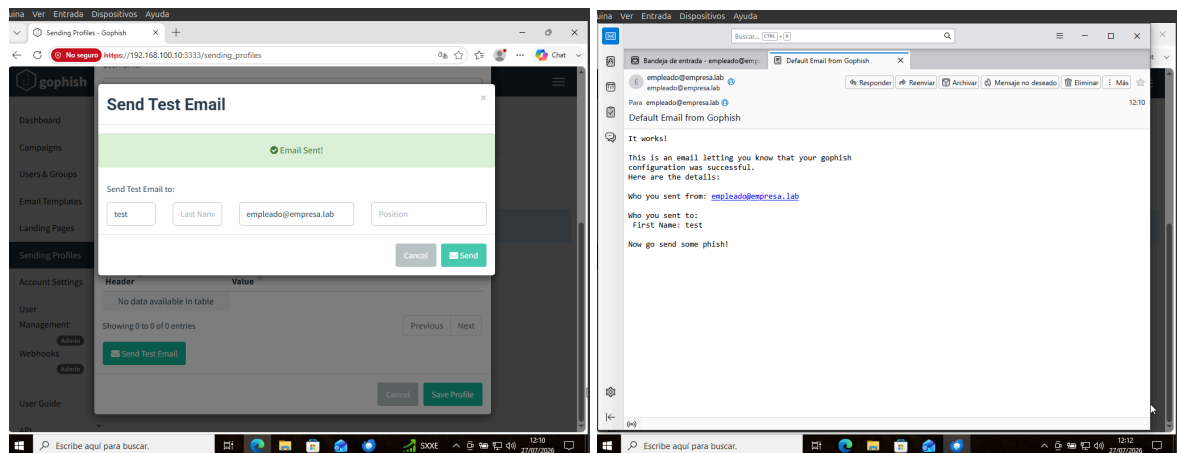
4.7.2 Configuración del perfil de envío (SMTP)

Una vez instalado GoPhish, el siguiente paso fue configurar el servidor de correo que utilizaría la aplicación para enviar las campañas de phishing.

Para ello se creó un nuevo perfil SMTP, utilizando el servidor Postfix configurado anteriormente.



Se indicó la dirección IP del servidor de correo, el puerto 25 y se configuró como remitente la cuenta empleado@empresa.lab, creada previamente.



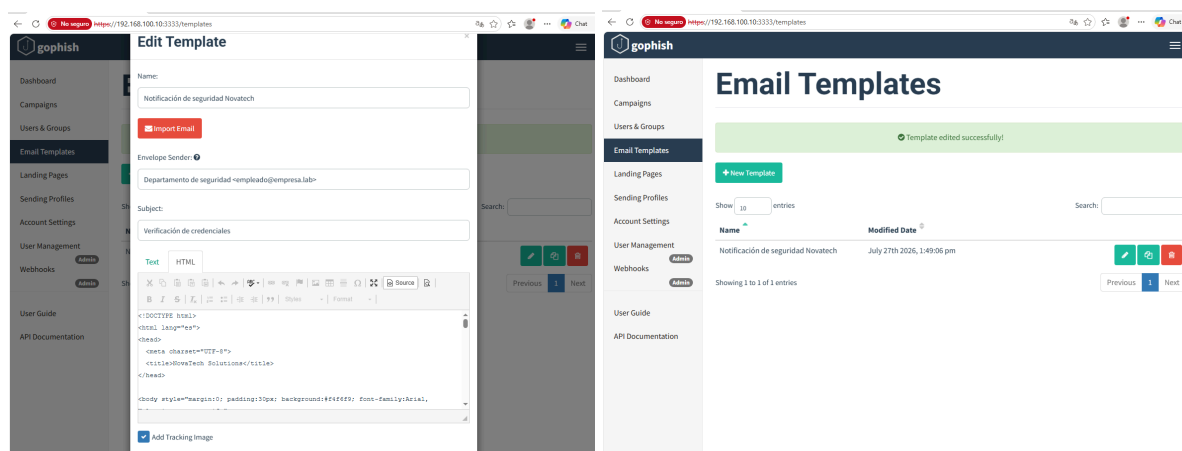
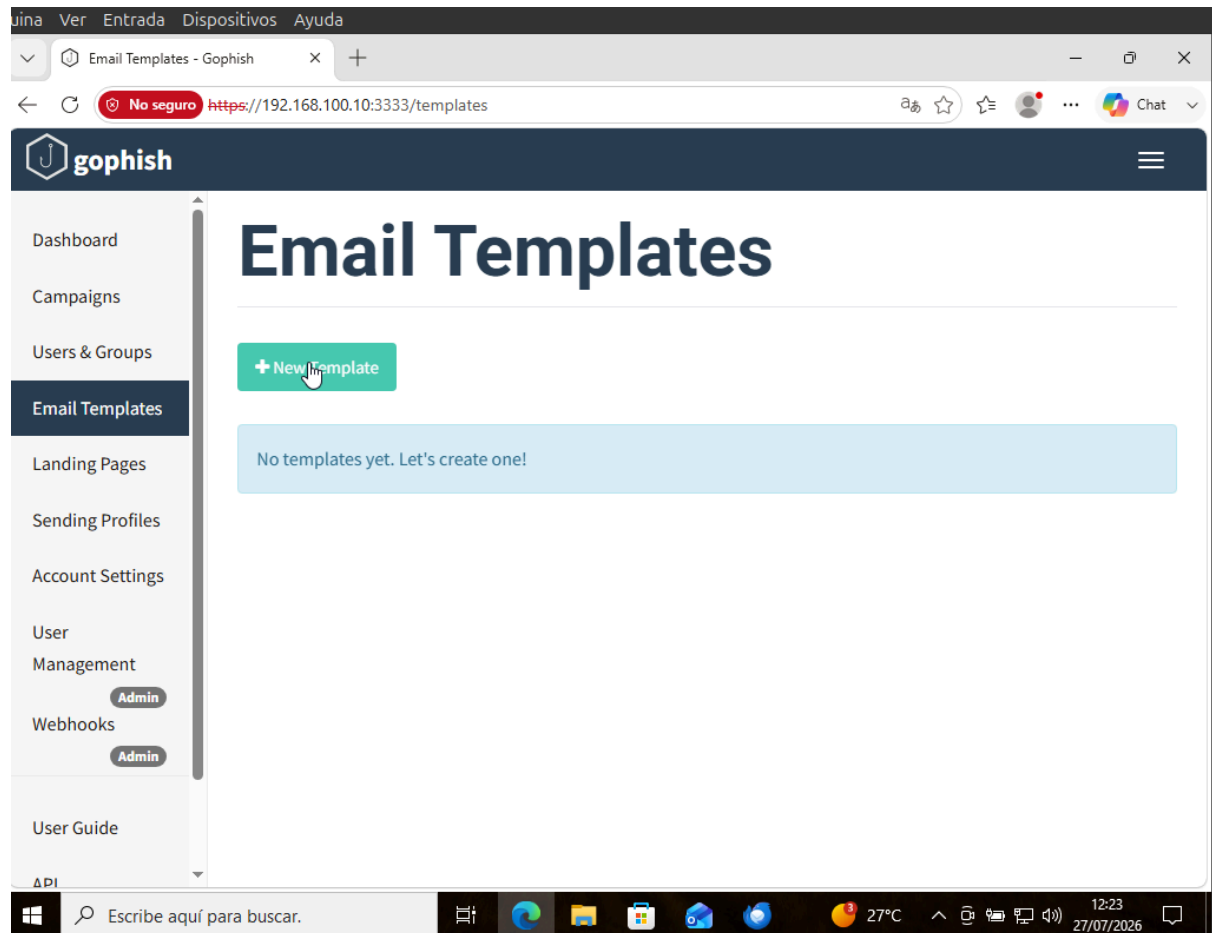
4.7.3 Creación de la plantilla de correo

Las plantillas de correo son uno de los elementos más importantes de GoPhish, ya que permiten crear los mensajes que recibirán los usuarios durante las campañas de phishing. En ellas se puede personalizar el asunto, el contenido del correo y utilizar variables que adaptan automáticamente el mensaje a cada destinatario.

Para este laboratorio se creó una plantilla con el aspecto de una notificación corporativa de la empresa ficticia NovaTech Solutions. Se utilizó un diseño sencillo, con un encabezado, colores corporativos y un botón que dirige al usuario a la página de phishing.

El correo informa de una supuesta verificación de credenciales y utiliza un lenguaje formal para hacer la simulación más creíble. Además, se emplearon variables de GoPhish, como {{.FirstName}} para mostrar el nombre del destinatario y {{.URL}} para insertar automáticamente el enlace de la campaña.

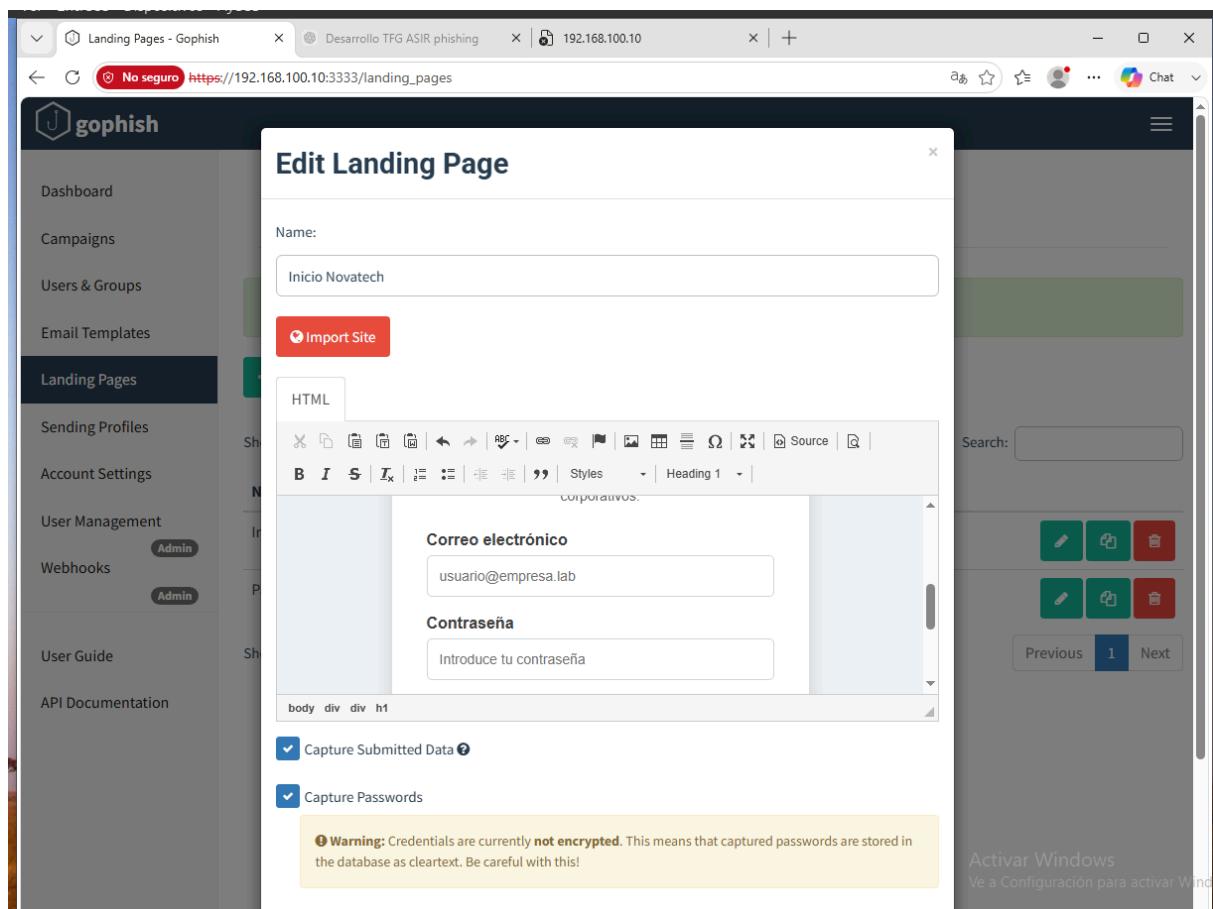
Una vez terminada, la plantilla quedó guardada en GoPhish para poder reutilizarla en futuras campañas.



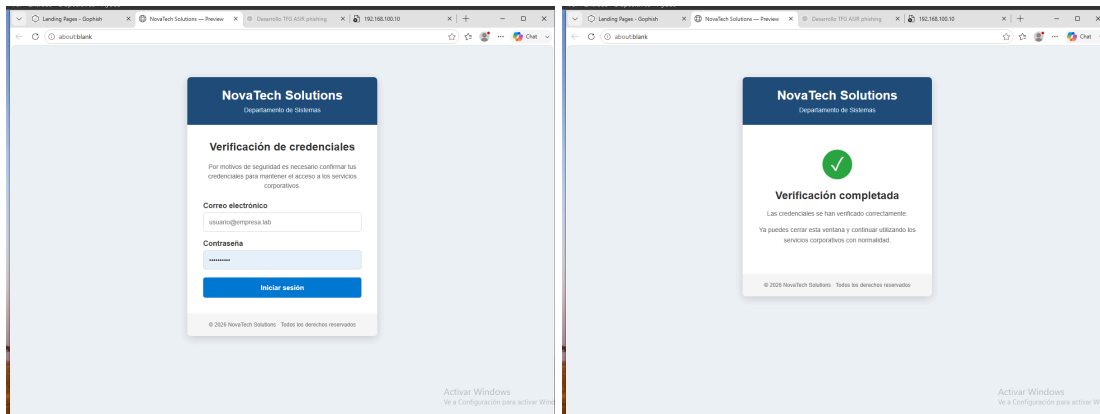
4.7.4 Creación de la página de phishing

Después de crear la plantilla de correo, se diseñó la página de phishing que vería el usuario al hacer clic en el enlace del mensaje. En GoPhish estas páginas se conocen como Landing Pages y permiten simular formularios de inicio de sesión.

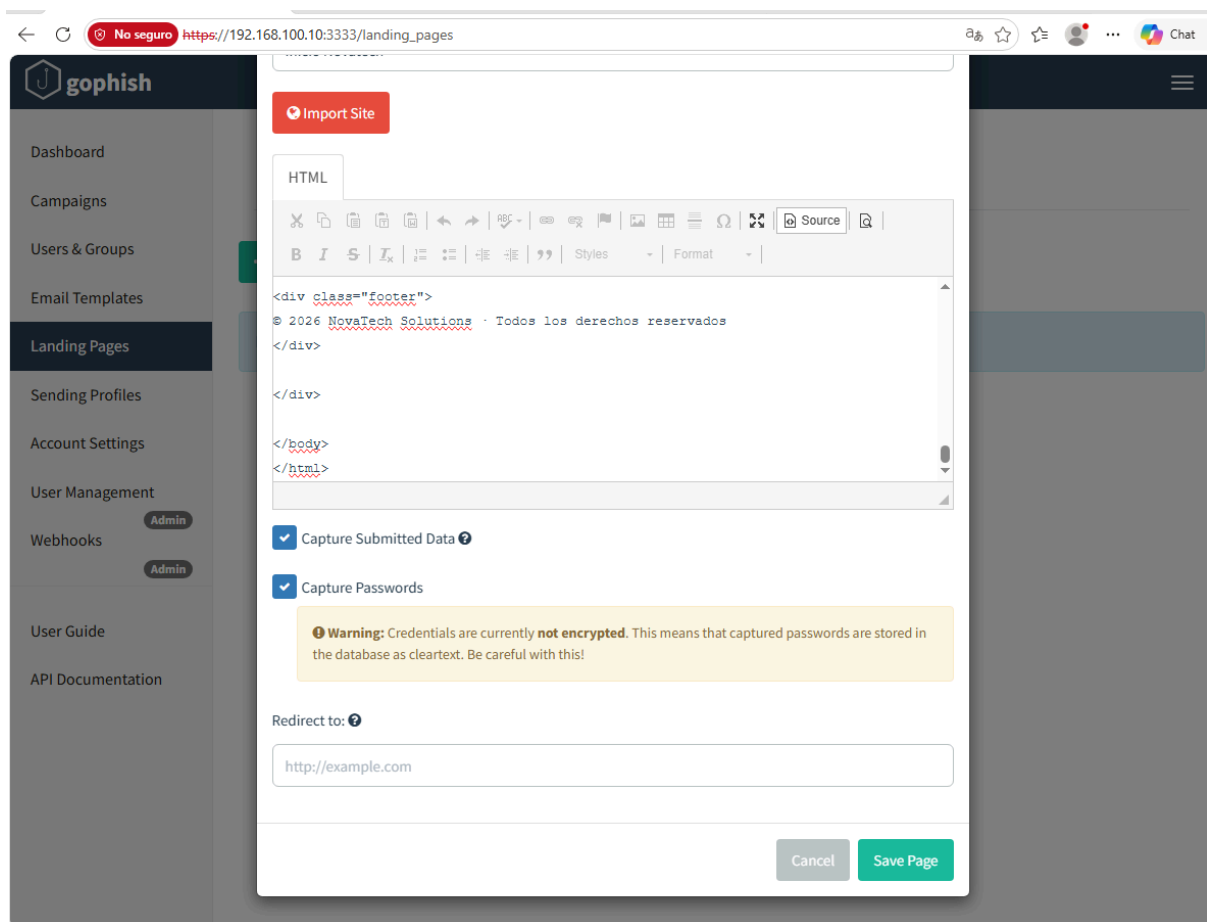
Para este laboratorio se creó una página con la imagen de la empresa ficticia NovaTech Solutions. En ella se solicita una dirección de correo electrónico y una contraseña, simulando un proceso de verificación de credenciales. Se utilizó un diseño sencillo y con el mismo estilo que el correo para dar una mayor sensación de continuidad.



Para que la simulación resultara más creíble, la página se diseñó con un pequeño comportamiento dinámico. Cuando el usuario pulsa el botón **Iniciar sesión**, se muestra durante unos segundos un mensaje de verificación acompañado de un indicador de carga. Una vez finalizado el proceso, aparece una confirmación indicando que la verificación se ha realizado correctamente.



Durante la configuración también se activaron las opciones de captura de datos enviados y captura de contraseñas de GoPhish

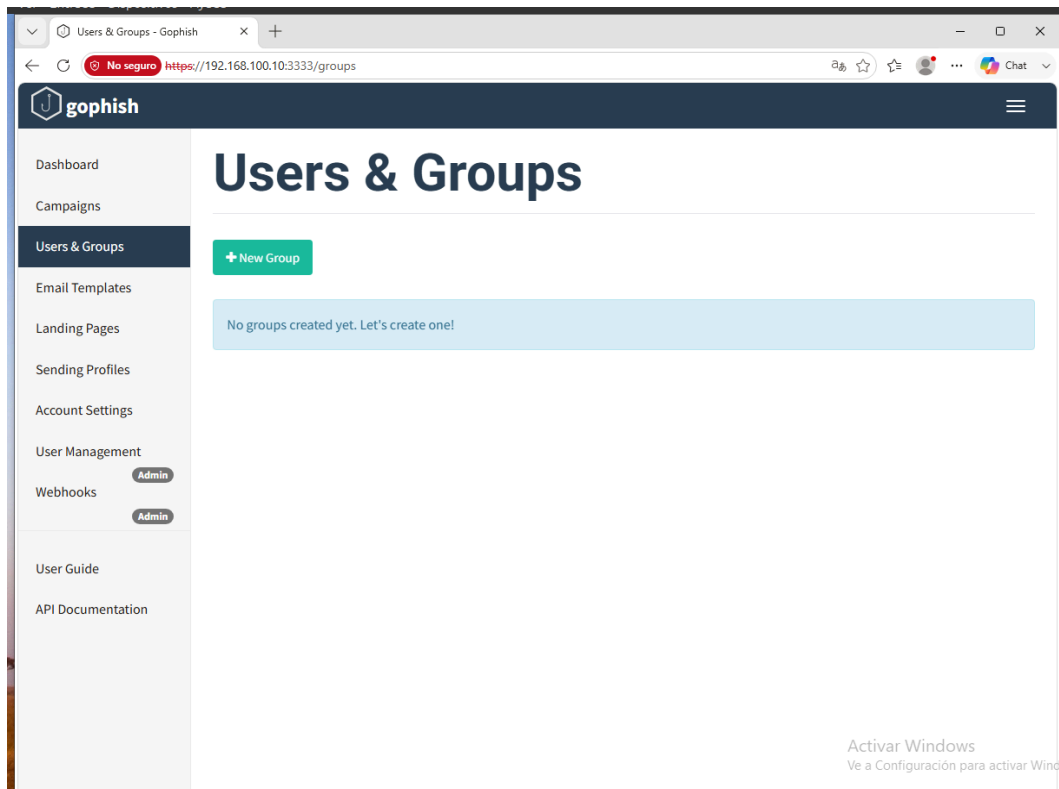


Una vez terminada la configuración, la página quedó guardada en GoPhish para utilizarla en la campaña de phishing. Cuando el usuario accede al enlace incluido en el correo electrónico, se muestra automáticamente esta página y la aplicación registra acciones como el acceso al formulario y el envío de las credenciales.

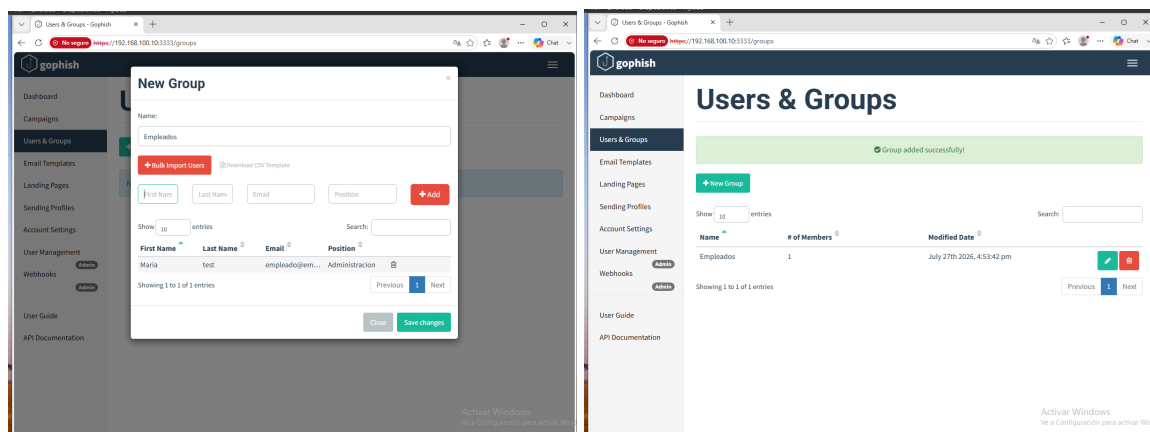
4.7.5 Creación del grupo de usuarios

Antes de crear la campaña fue necesario definir el grupo de usuarios al que se enviarían los correos de phishing. En GoPhish, los destinatarios se organizan en grupos, lo que facilita su selección a la hora de lanzar una campaña.

En este laboratorio se creó un grupo con un único usuario de prueba, suficiente para comprobar que todo el proceso funcionaba.

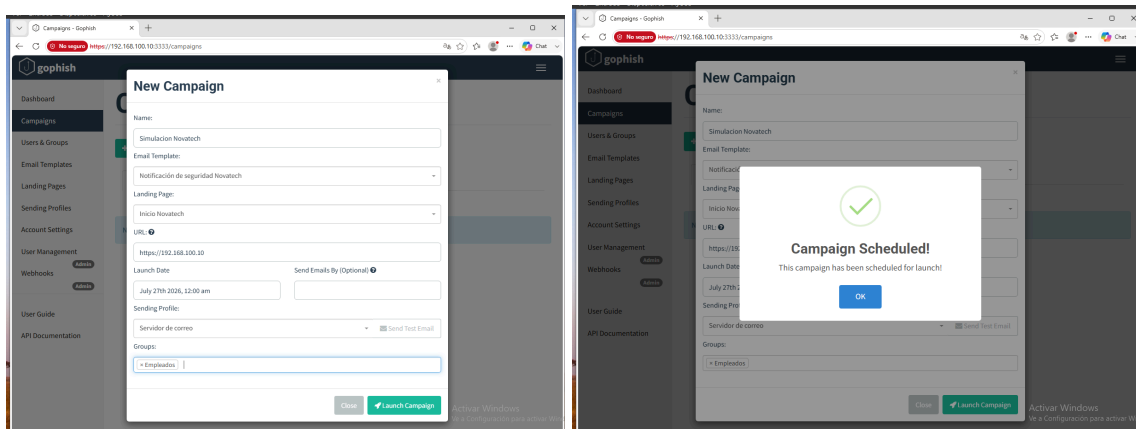


Durante la configuración se introdujo el nombre del usuario, su dirección de correo empleado@empresa.lab y otros datos, como el nombre y los apellidos, que GoPhish puede utilizar para personalizar automáticamente los mensajes.

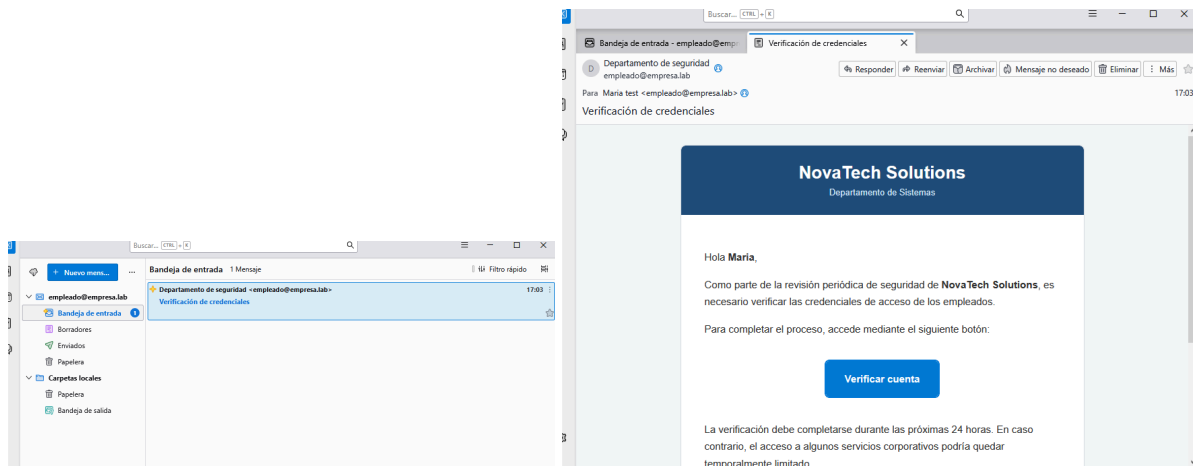


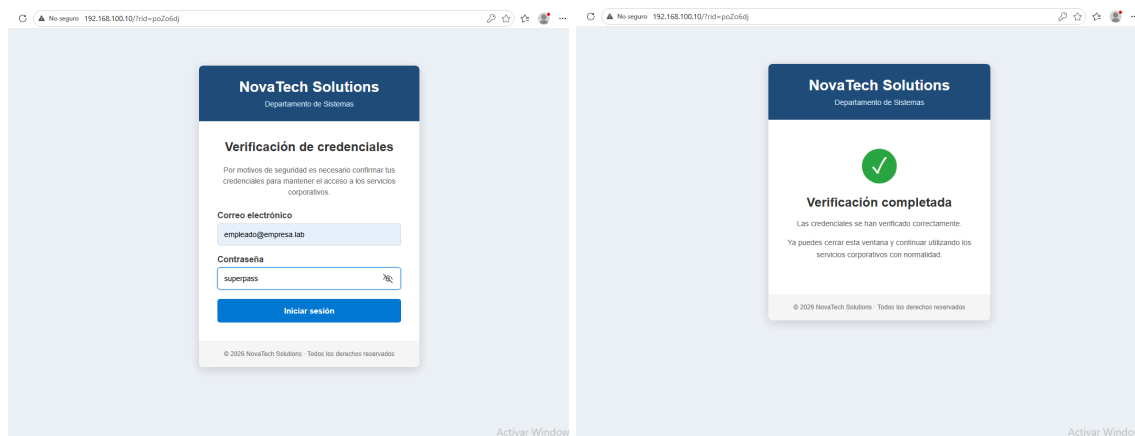
4.7.6 Creación de la campaña y prueba de funcionamiento

Una vez configurados todos los elementos necesarios, el siguiente paso fue crear la campaña de phishing, para ello se asignó un nombre a la campaña y se seleccionaron el perfil SMTP, la plantilla de correo, el grupo de usuarios y la página de phishing creados en los apartados anteriores. También se configuró la dirección web que recibiría el usuario al pulsar el enlace del correo, correspondiente a la página de phishing alojada en el servidor.

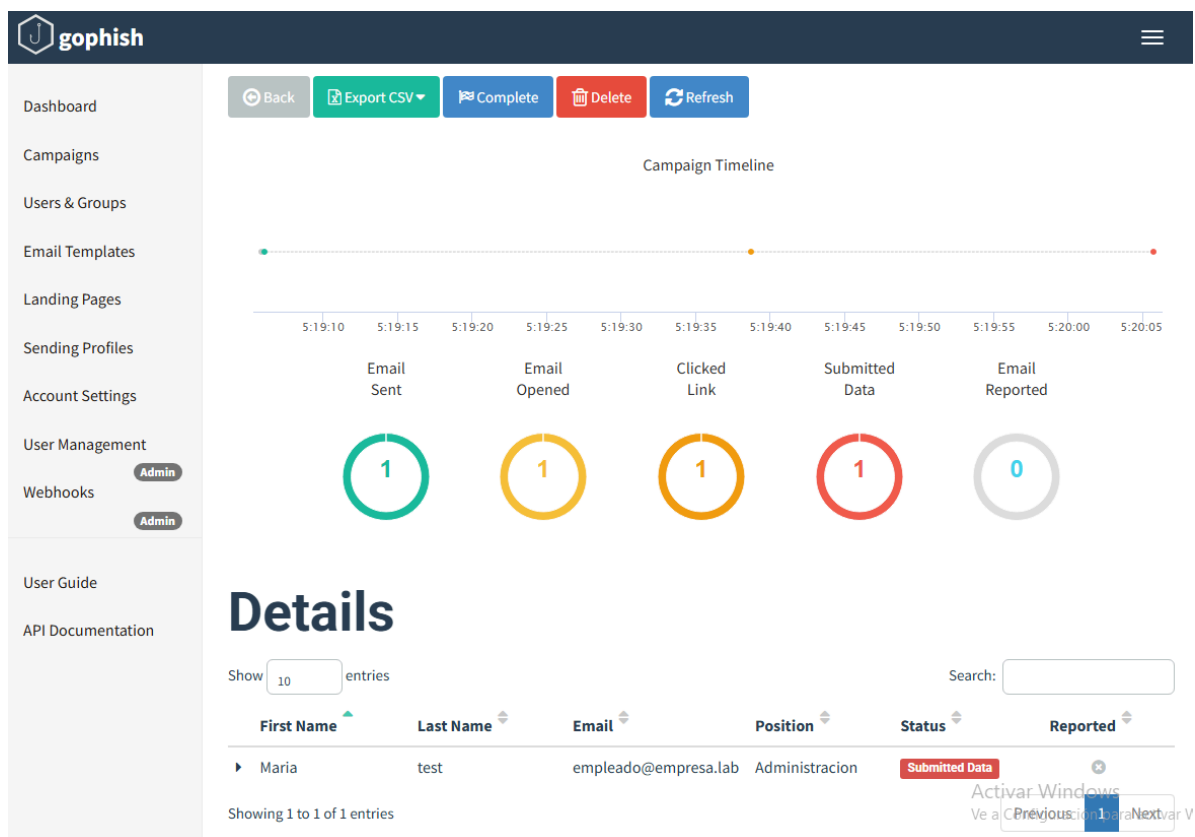


Cuando el usuario accede al enlace recibido por correo, se muestra la página de inicio de sesión creada para la simulación. Después de introducir sus credenciales y pulsar Iniciar sesión, aparece un breve proceso de verificación con un indicador de carga. Al finalizar, se muestra un mensaje indicando la verificación, haciendo que resulte más realista.





Mientras tanto, GoPhish registra automáticamente las acciones realizadas por el usuario durante la campaña. De este modo, es posible comprobar si el correo ha sido abierto, si el usuario ha accedido al enlace y si ha enviado las credenciales introducidas en el formulario.



Dashboard

Campaigns

Users & Groups

Email Templates

Landing Pages

Sending Profiles

Account Settings

User Management

Webhooks

User Guide

API Documentation

Timeline for Maria test

Email: empleado@empresa.lab

Result ID: 7uh98IG

Campaign Created

July 27th 2026 5:19:05 pm

Email Sent

July 27th 2026 5:19:06 pm

Clicked Link

July 27th 2026 5:19:38 pm

Windows (OS Version: 10)

Chrome (Version: 150.0.0.0)

Submitted Data

July 27th 2026 5:20:05 pm

Windows (OS Version: 10)

Chrome (Version: 150.0.0.0)

Replay Credentials

View Details

Parameter	Value(s)
email	empleado@empresa.lab
password	superpass

Showing 1 to 1 of 1 entries

5. Conclusiones

5.1 Objetivos alcanzados

El objetivo principal de este trabajo era diseñar e implementar un laboratorio de simulación de phishing en un entorno controlado que permitiera entender cómo funciona este tipo de ataque sin poner en riesgo equipos o usuarios reales.

Para conseguirlo, se configuraron todos los elementos necesarios para simular una campaña de phishing completa. Se instaló un servidor con GoPhish, un servidor de correo y un equipo cliente, todos conectados mediante una red interna aislada. Además, se creó una plantilla de correo, una página de inicio de sesión simulada y una campaña de phishing totalmente funcional.

Las pruebas realizadas demostraron que el laboratorio funciona correctamente. El usuario recibió el correo, pudo acceder a la página de phishing y GoPhish registró sin problemas las acciones realizadas durante la simulación, como la apertura del enlace y el envío de las credenciales. Gracias a ello, se puede afirmar que se han alcanzado los objetivos planteados al inicio del proyecto.

5.2 Conocimientos adquiridos

La realización de este proyecto me ha permitido conocer mejor cómo funciona una campaña de phishing y entender todas las fases que intervienen en este tipo de ataques. Durante el desarrollo del laboratorio he aprendido a trabajar con herramientas como GoPhish, Postfix, Dovecot, Thunderbird y Ubuntu Server, además de mejorar mis conocimientos sobre administración de sistemas Linux, configuración de redes e integración de diferentes servicios en un mismo entorno.

5.3 Trabajo futuro

Aunque el laboratorio cumple los objetivos planteados, todavía ofrece muchas posibilidades de mejora. Una de ellas sería crear nuevos escenarios de phishing e incorporar diferentes técnicas de ingeniería social para simular situaciones más variadas.

También sería interesante integrar un sistema SIEM, como Wazuh, para recopilar y analizar los eventos generados durante las campañas. De esta forma, además de simular el ataque, también sería posible estudiar cómo detectarlo y analizarlo desde el punto de vista defensivo.

Por último, el laboratorio podría ampliarse con nuevas plantillas de correo, diferentes páginas de phishing y campañas dirigidas a varios usuarios. Esto permitiría recrear situaciones más parecidas a las que pueden encontrarse en un entorno empresarial.